

Introduction to VS Code

SISMID Lab

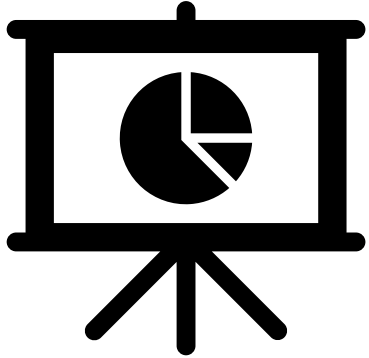
July 22 through July 24, 2026

A Human-AI Teaming Approach to Infectious Disease Modeling

Lecture Overview

Topic	Slides
1. What is Visual Studio Code?	6
2. VS Code Layout	7–11
3. Installing the R Extension	12–15
4. Opening an R Script	16–17
5. GitHub Copilot Setup	18–22
6. GitHub Copilot Layout	23–31
7. Connecting GitHub & VS Code	32–43
8. Preparing the Working Directory	44–47
9. Setting the Working Directory in VS Code	48–52

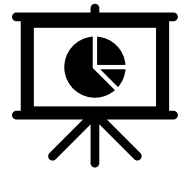
General Housekeeping



Any slide with this icon contains the **overview** and **expected outputs** for the included tasks. All outputs listed on these slides should be uploaded to the course GitHub repository for review.

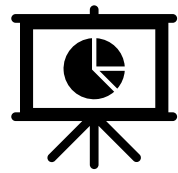


Any slide with this icon indicates that it is **time to complete a hands-on activity**. Carefully follow the instructions provided, complete the required task, and verify your results.



Task Overviews

Task	Description
Task 1: Installing the R Extension	You'll add R support to VS Code by opening the Extensions Marketplace, searching for REditorSupport, and installing it so you can write and run R code.
Task 2: GitHub Copilot Setup	You'll turn on GitHub Copilot, VS Code's built-in AI helper, by signing in with your GitHub account and authorizing VS Code.
Task 3: Connecting GitHub & VS Code	You'll bring your forked project into VS Code by checking that Git is installed, copying your repo's link, cloning it, and opening it in the Explorer.
Task 4: Preparing the Working Directory	You'll set up your workspace by opening your cloned GitHub folder, going into the <i>USE-ME</i> folder, and creating a folder with your name to work in.
Task 5: Setting the Working Directory in VS Code	You'll tell VS Code which folder to work in by closing the current folder, opening your named folder, and confirming it's the active one.

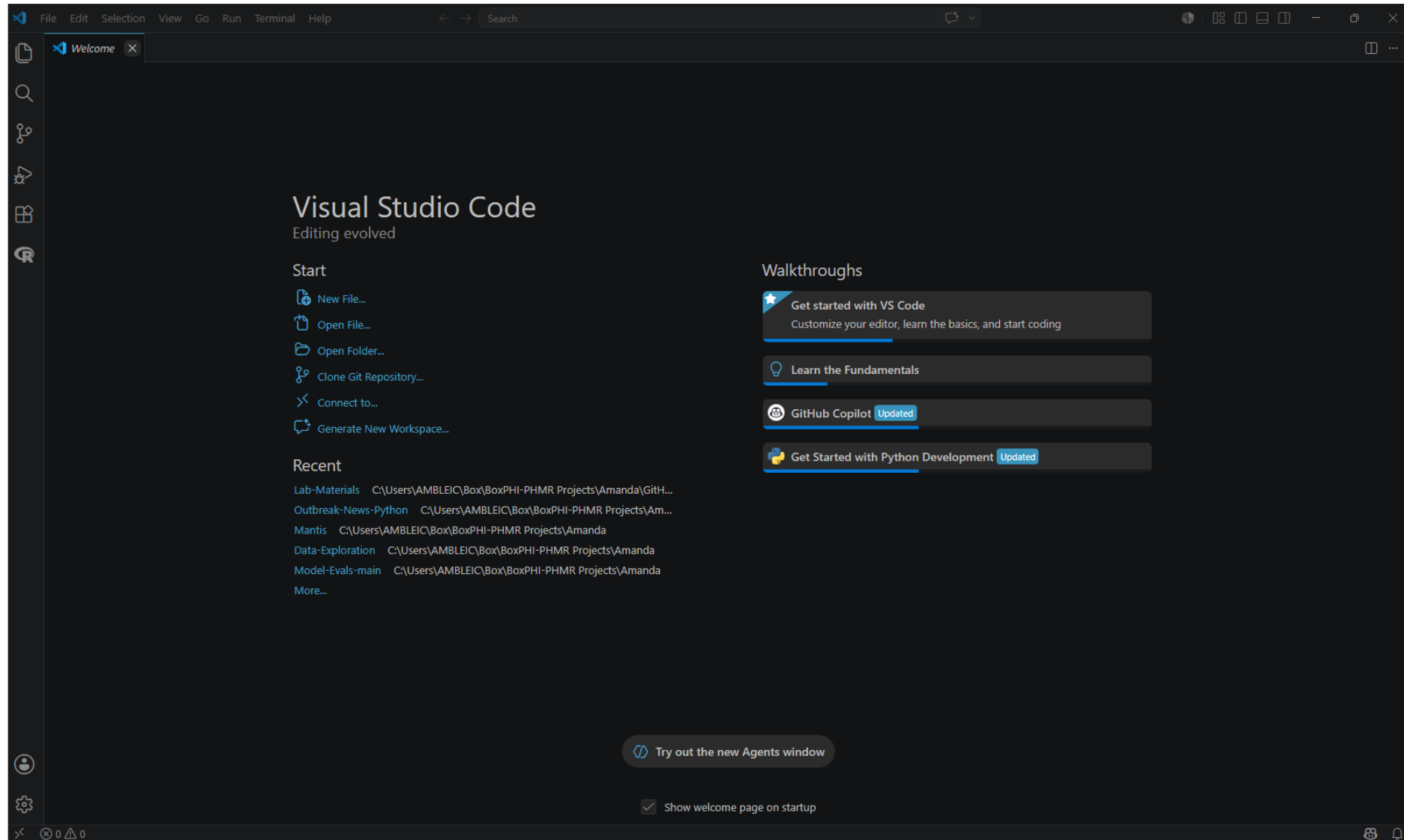


Goal Output

By the end of the lab, you should have your own **fork** of the course repository **cloned into VS Code**, plus your own **working directory** set up and ready to go.

Activity Output	Description
Cloned GitHub fork	A copy of your forked repository downloaded into VS Code, so you can open, view, and edit all of the course files right on your own computer.
Working directory	Your own named folder inside the <i>USE-ME</i> folder, set as VS Code's working directory so everything you create is saved in the right place.

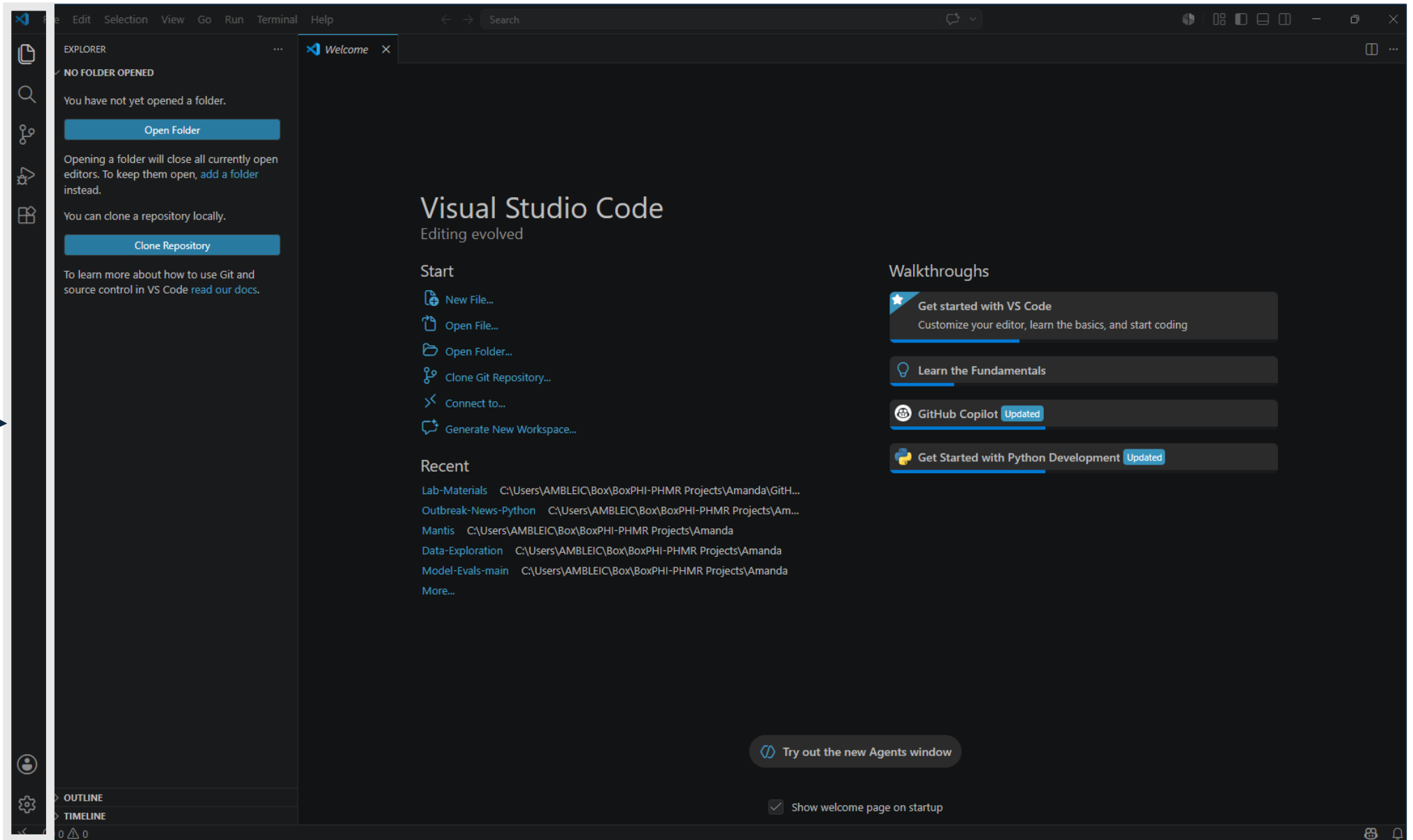
Visual Studio Code (VS Code)



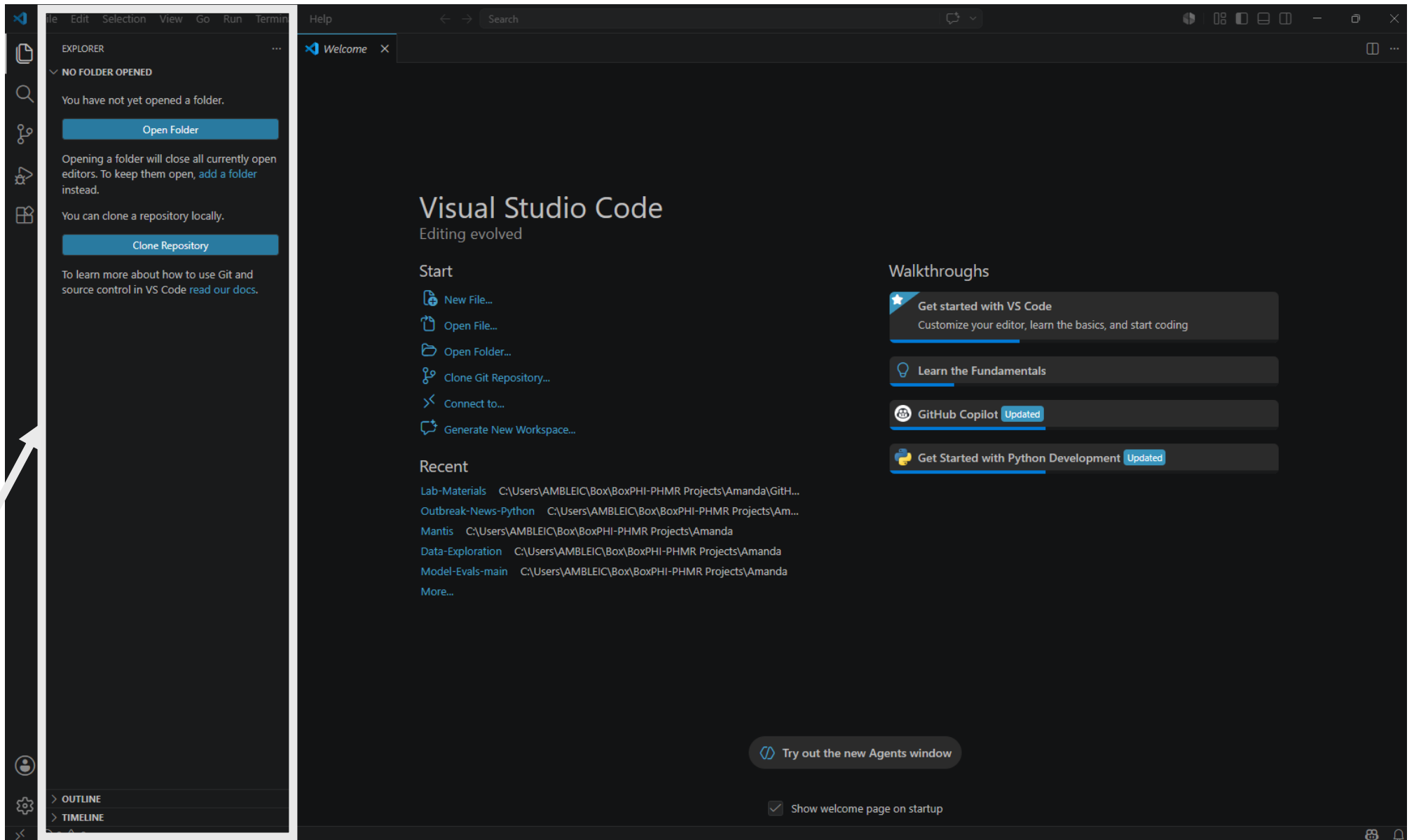
VS Code Layout

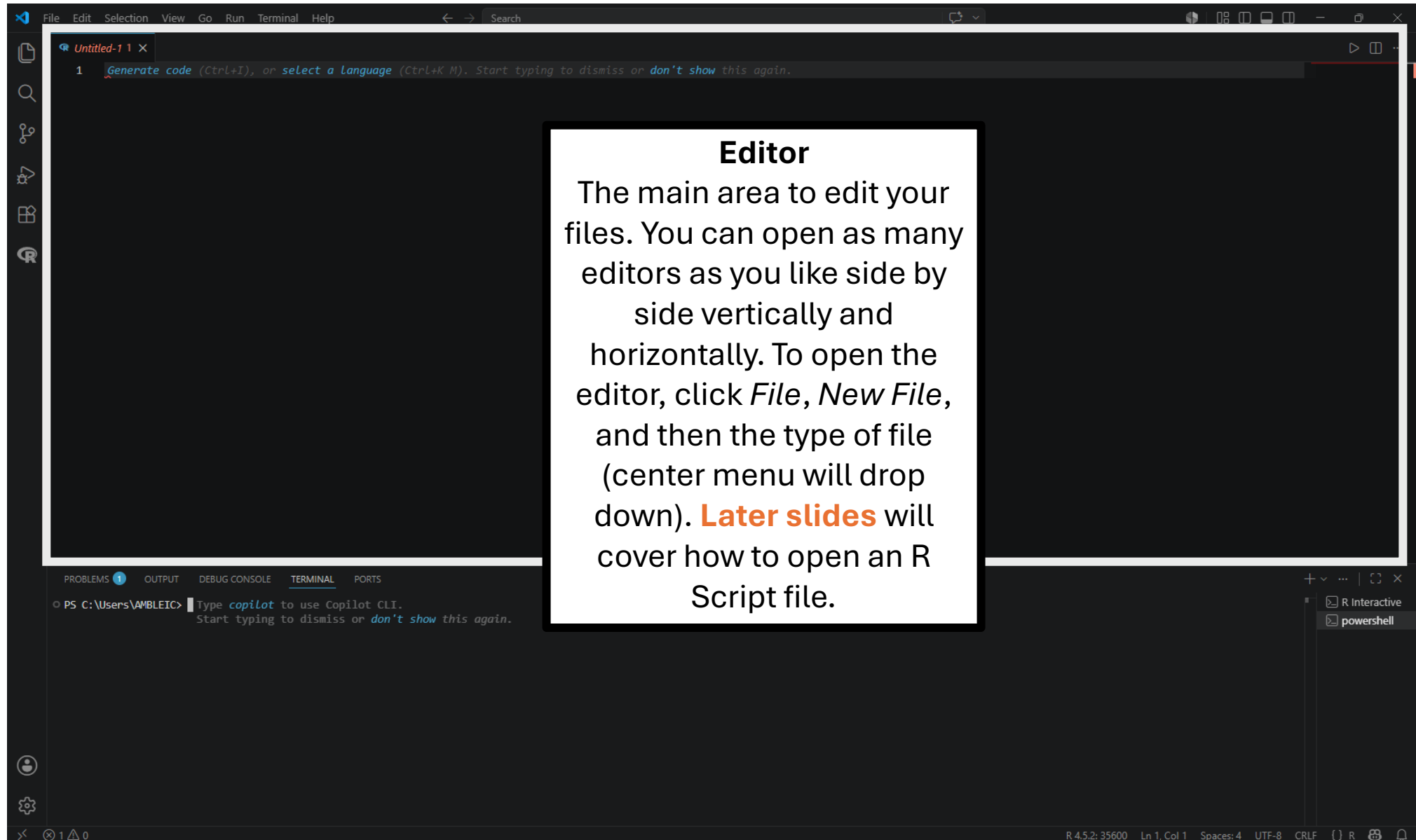
Activity Bar

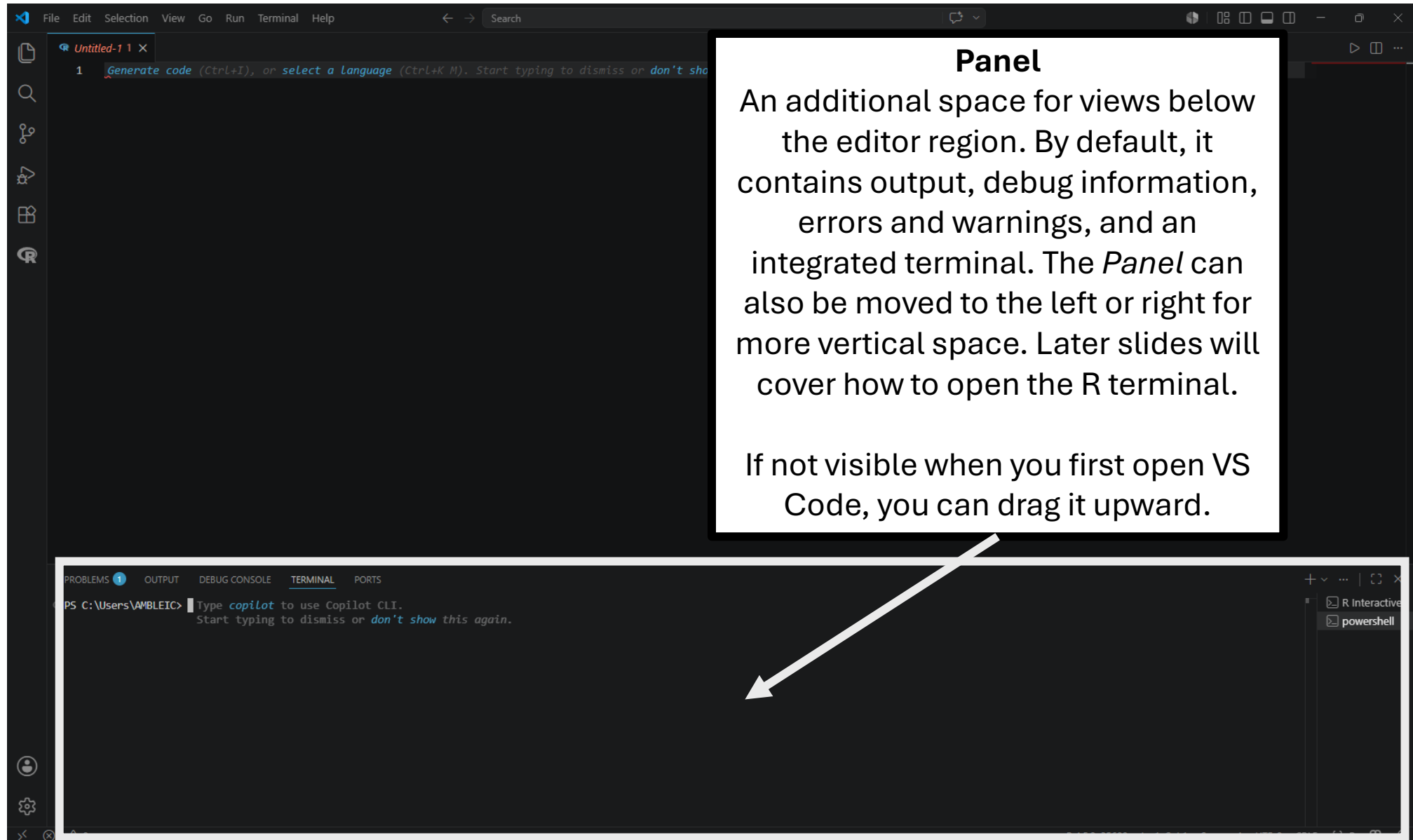
Located on the far left-hand side. Lets you switch between views and gives you additional context-specific indicators, like the number of outgoing changes when Git is enabled. You can change the position of the Activity Bar.



Primary Side Bar
Contains different views like the Explorer to assist you while working on your project. Later slides will focus on how to set up the working directory here.







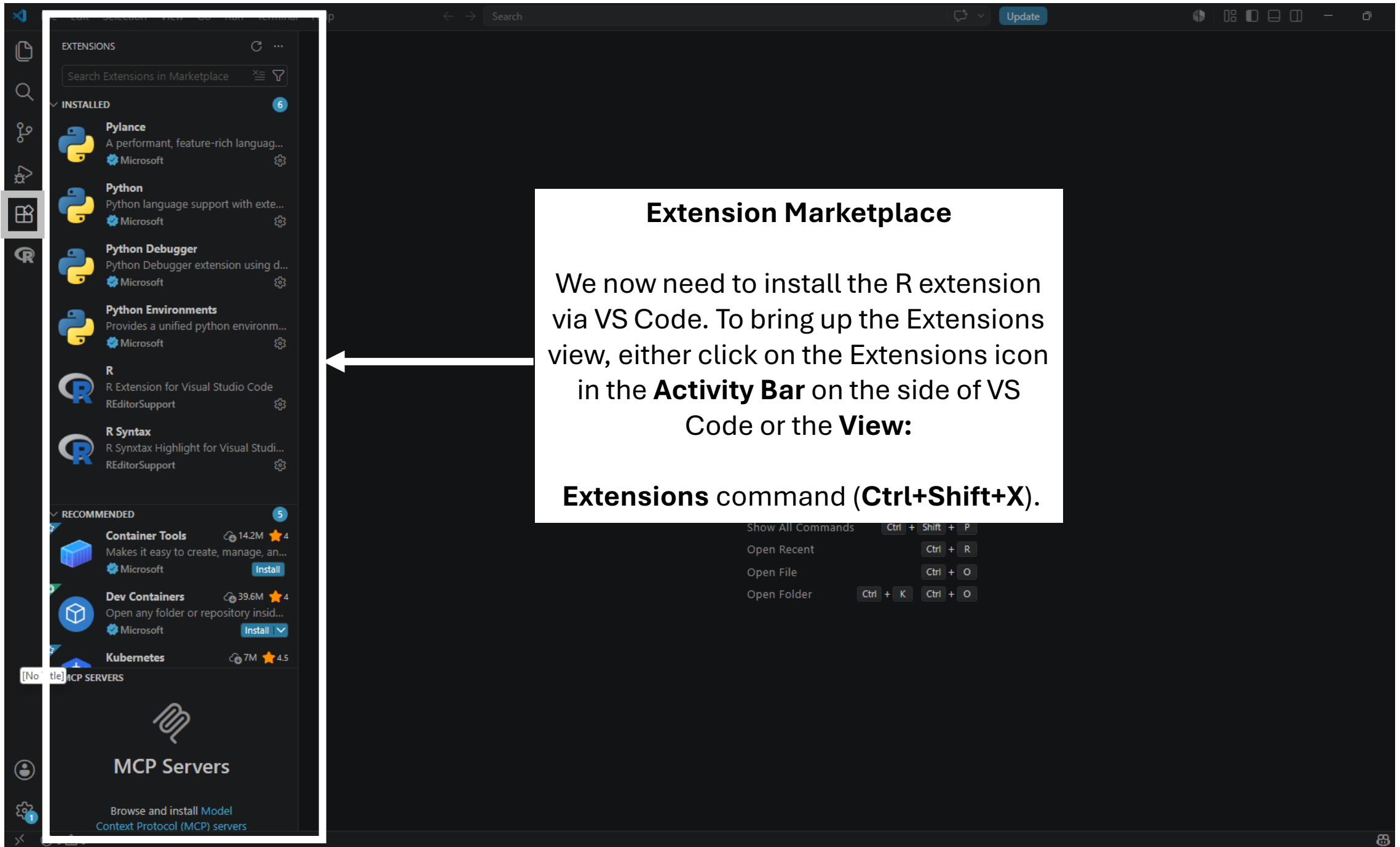
Panel

An additional space for views below the editor region. By default, it contains output, debug information, errors and warnings, and an integrated terminal. The *Panel* can also be moved to the left or right for more vertical space. Later slides will cover how to open the R terminal.

If not visible when you first open VS Code, you can drag it upward.



Task 1: Installing the R Extension



Extension Marketplace

We now need to install the R extension via VS Code. To bring up the Extensions view, either click on the Extensions icon in the **Activity Bar** on the side of VS Code or the **View:**

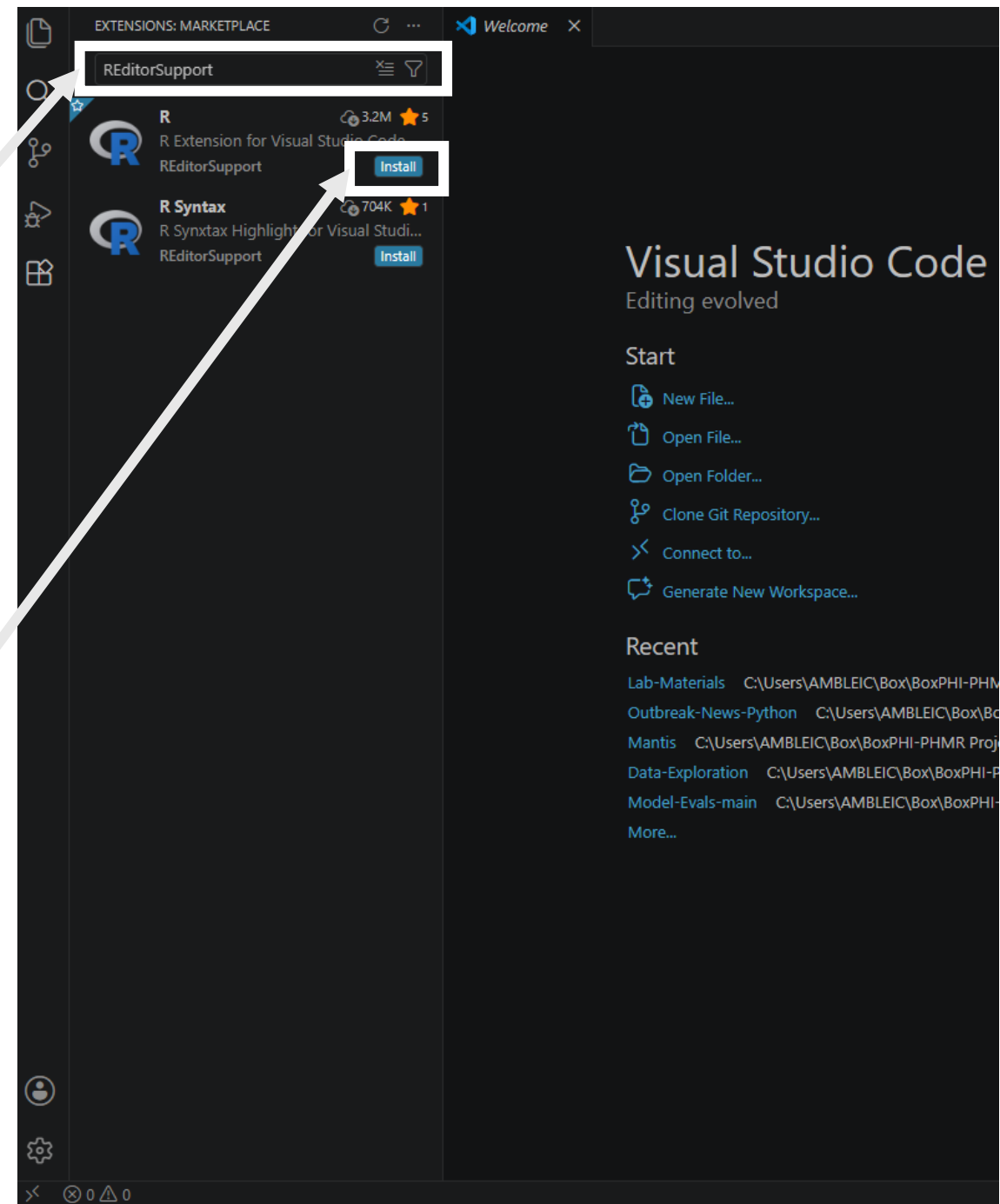
Extensions command (**Ctrl+Shift+X**).

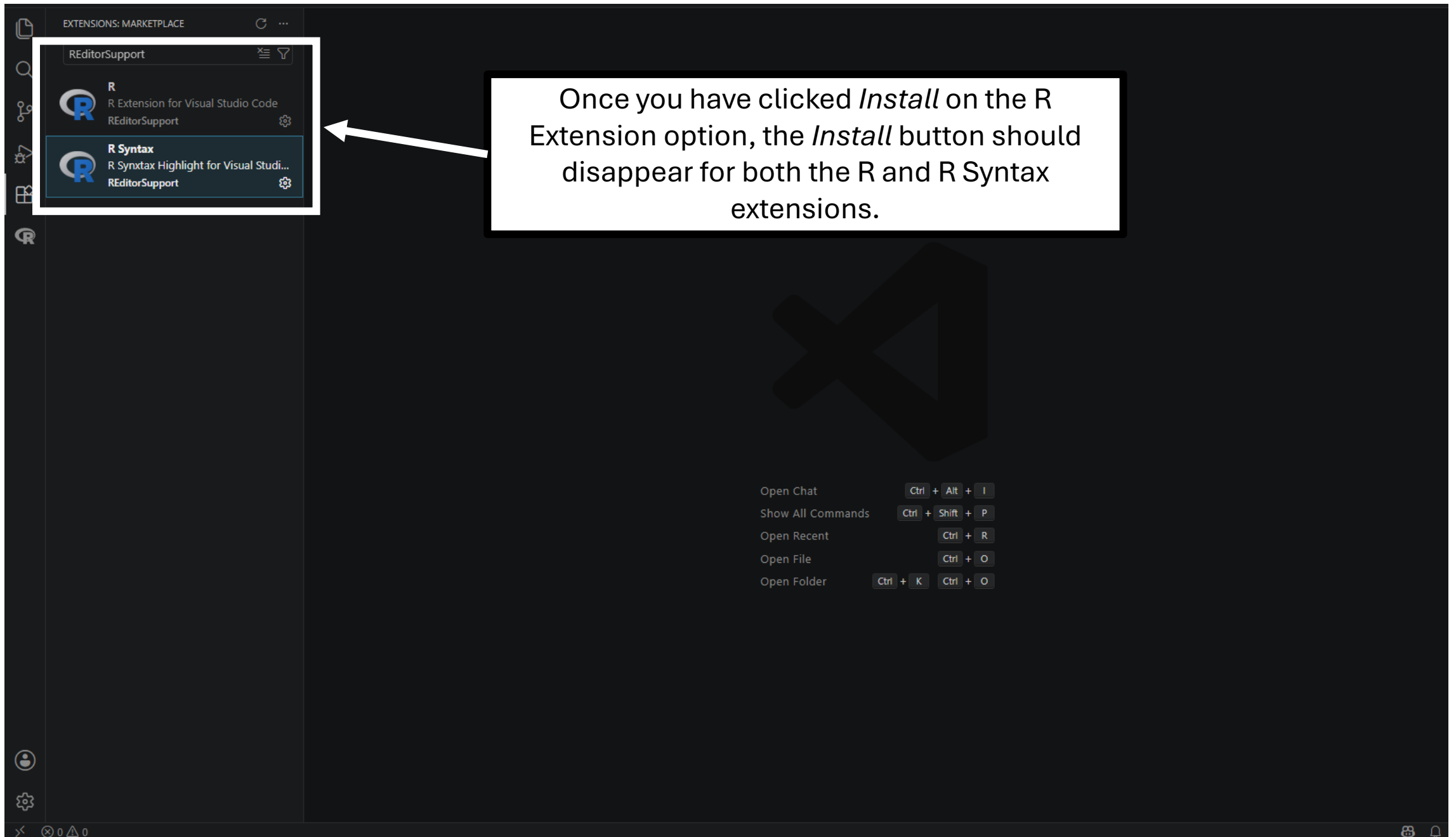
Step 1: Search for REditorSupport

After you have the *Extensions Marketplace* open, you will then want to search for *REditorSupport*. This will connect R and VS Code.

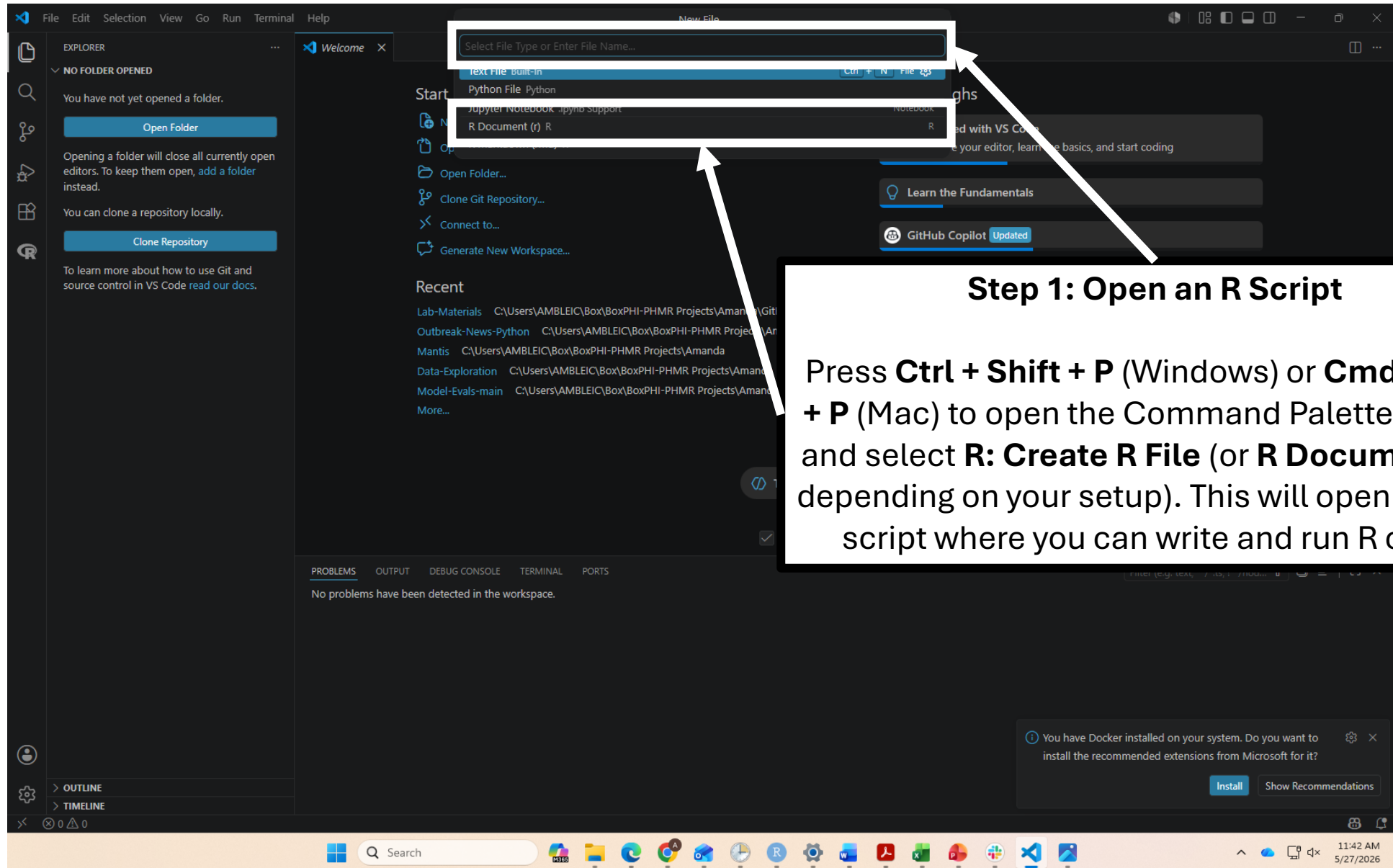
Step 2: Install R Extension

By this point, you should have R downloaded onto your local system. If you have not done so, please navigate to <https://cran.r-project.org> and download R. If you have R downloaded, all you need to do for this step is click the *Install* button for the R Extension.





Opening an R Script



Step 1: Open an R Script

Press **Ctrl + Shift + P** (Windows) or **Cmd + Shift + P** (Mac) to open the Command Palette. Type **R** and select **R: Create R File** (or **R Document (R)**, depending on your setup). This will open a new R script where you can write and run R code.

The image shows a screenshot of the RStudio application. At the top, there is a menu bar with 'Terminal' and 'Help' options. Below the menu bar is a toolbar with various icons. The main window is divided into several panes. The top pane, titled 'Untitled-1', contains a single line of code: `1 Generate code (Ctrl+I), or select a language (Ctrl+K M). Start typing to dismiss or don't show this again.`. The bottom pane is the 'TERMINAL' pane, which displays the R welcome message. The message includes information about R being free software, a collaborative project, and instructions on how to use R, including running demos, getting help, and quitting. The terminal prompt is `>` followed by a cursor. On the right side of the terminal pane, there is a list of open terminals: 'powershell' and 'R Interactive'. The 'R Interactive' terminal is selected.

Step 2: Open the R Terminal

Press **Ctrl + Enter** to run a line of code, which opens the R terminal (i.e., where we can see any errors and the log of our code runs). Most of the work for this lab will happen within the R scripts, rather than the R terminal.

```
R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.

Natural language support but running in an English locale

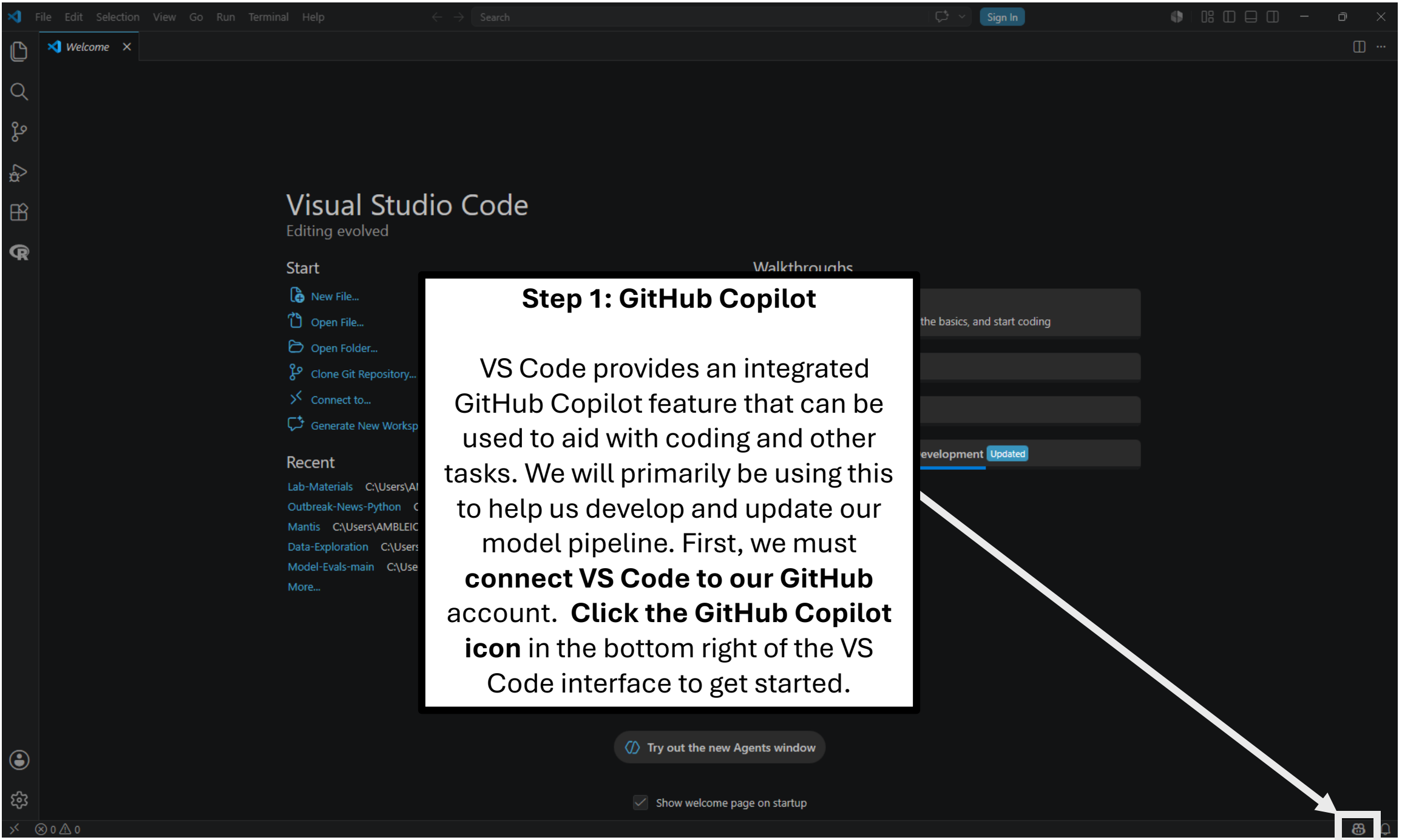
R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.

Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.

>
> |
```

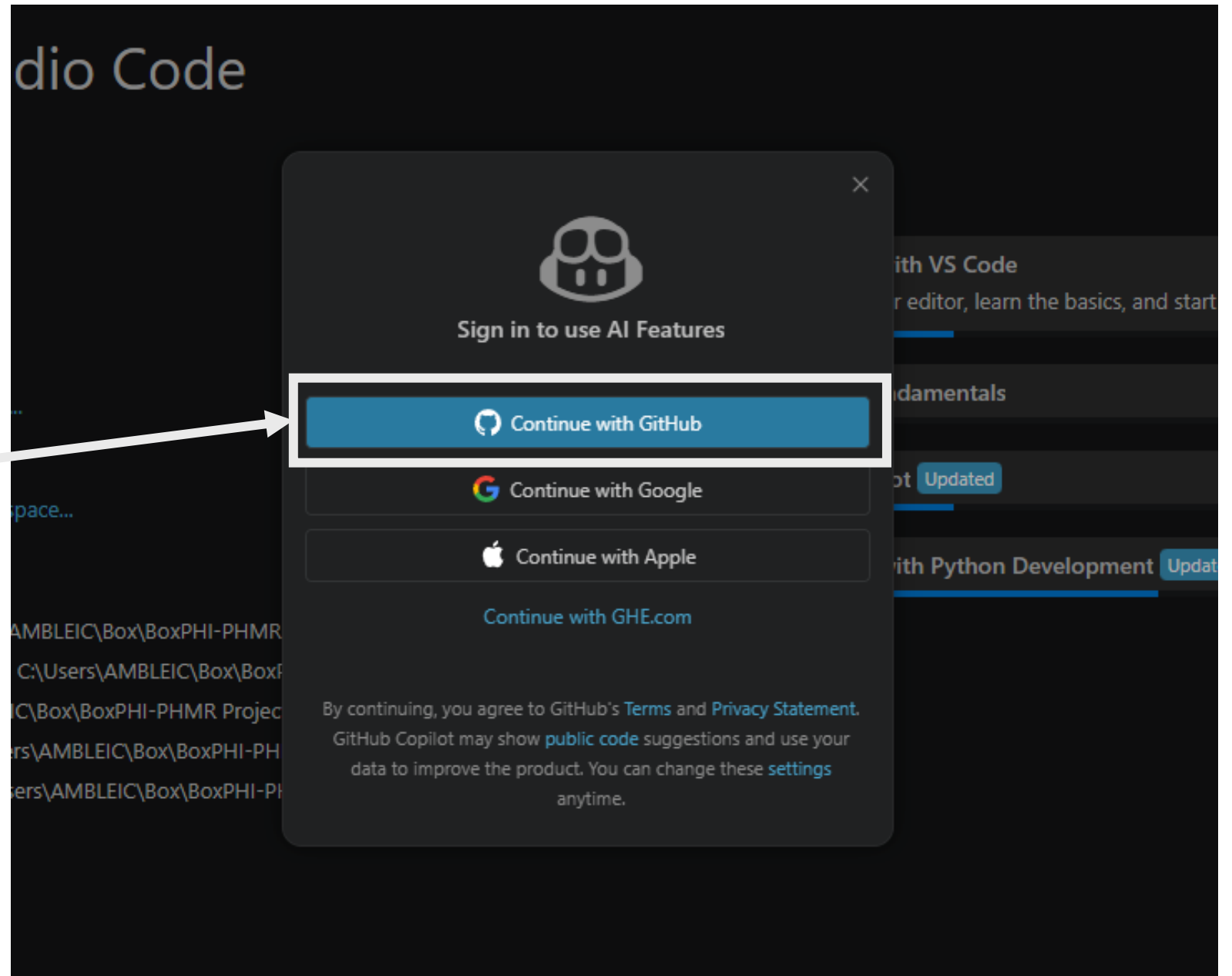


Task 2: GitHub Copilot Setup



Step 2: Continue with GitHub

Next, click the “Continue with GitHub” option. This will open the GitHub login page and is where you will log into your GitHub account.





Authorize Visual Studio Code

From the options below, choose which account you would like to use to authorize this app.

	Signed in as bleicham	Continue
	clemson-dma-prime Signed out	Sign in

or

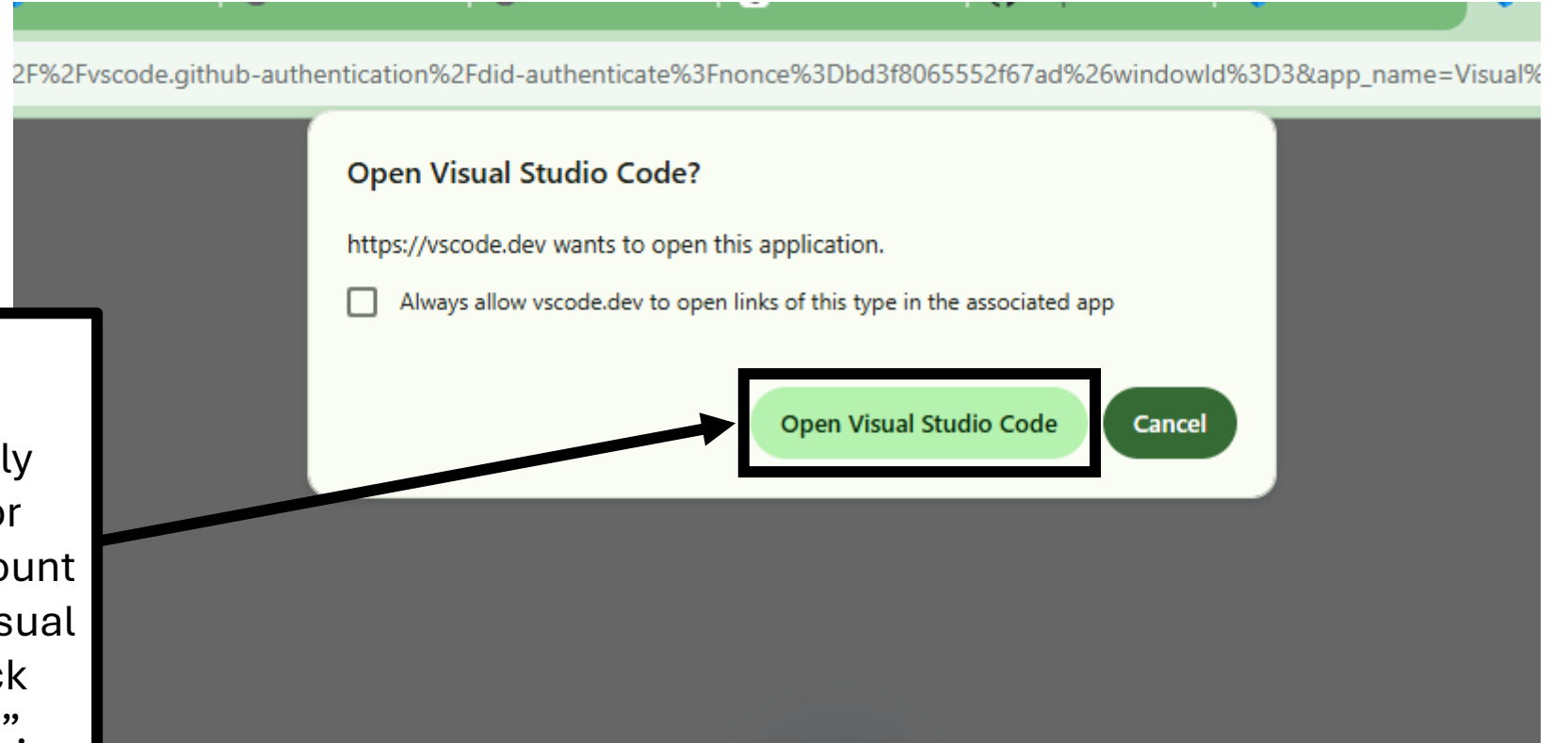
Use a different account

Step 3: Authorize VS Code

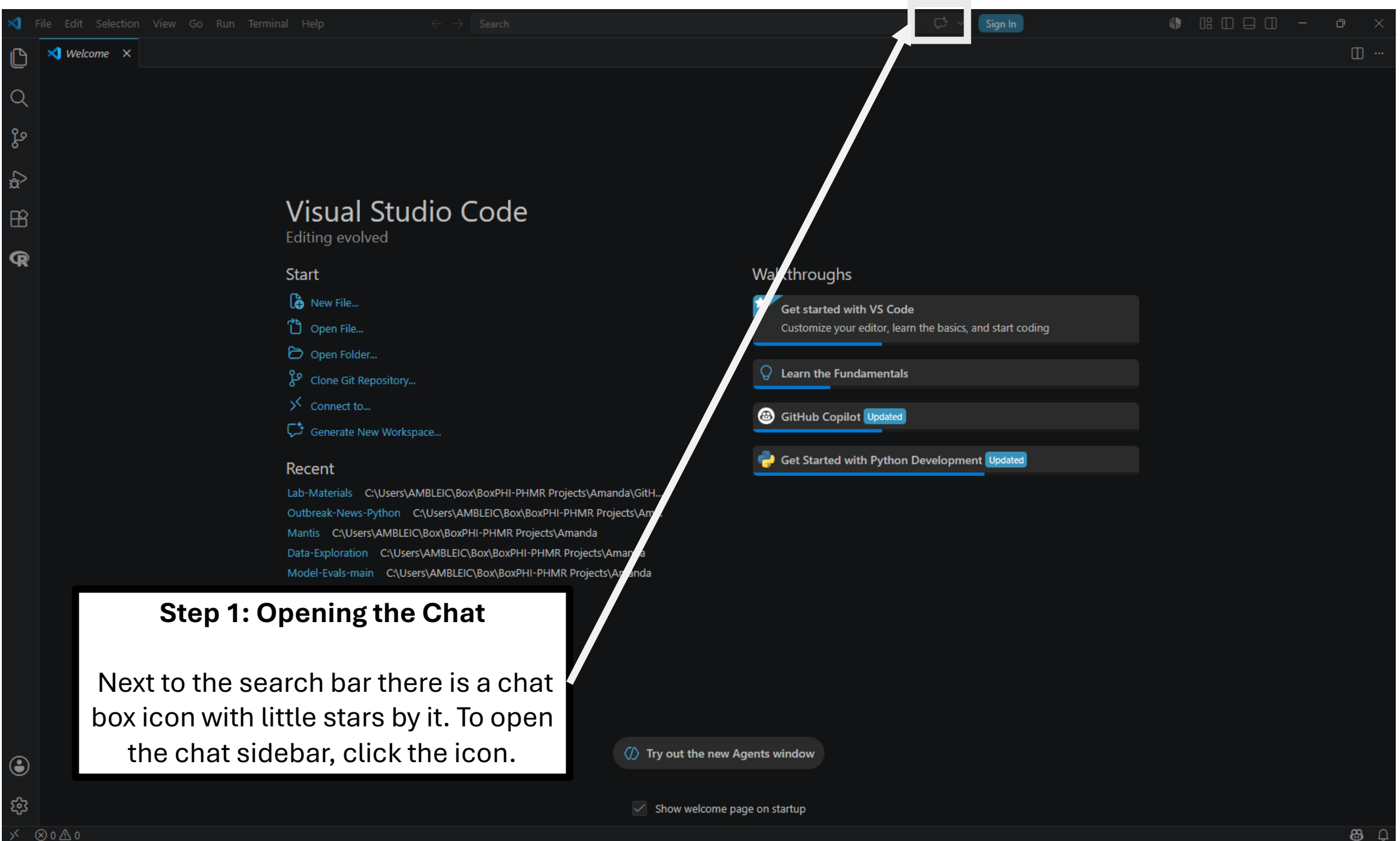
If you are already logged into your GitHub account, click “Continue”. Otherwise, log into your GitHub account and then authorize VS Code.

Step 4: Open VS Code

Once you have successfully authorized VS Code and/or logged into your GitHub account you should see the “Open Visual Studio Code” pop-up. Click “Open Visual Studio Code”.



GitHub Copilot Layout



Step 1: Opening the Chat

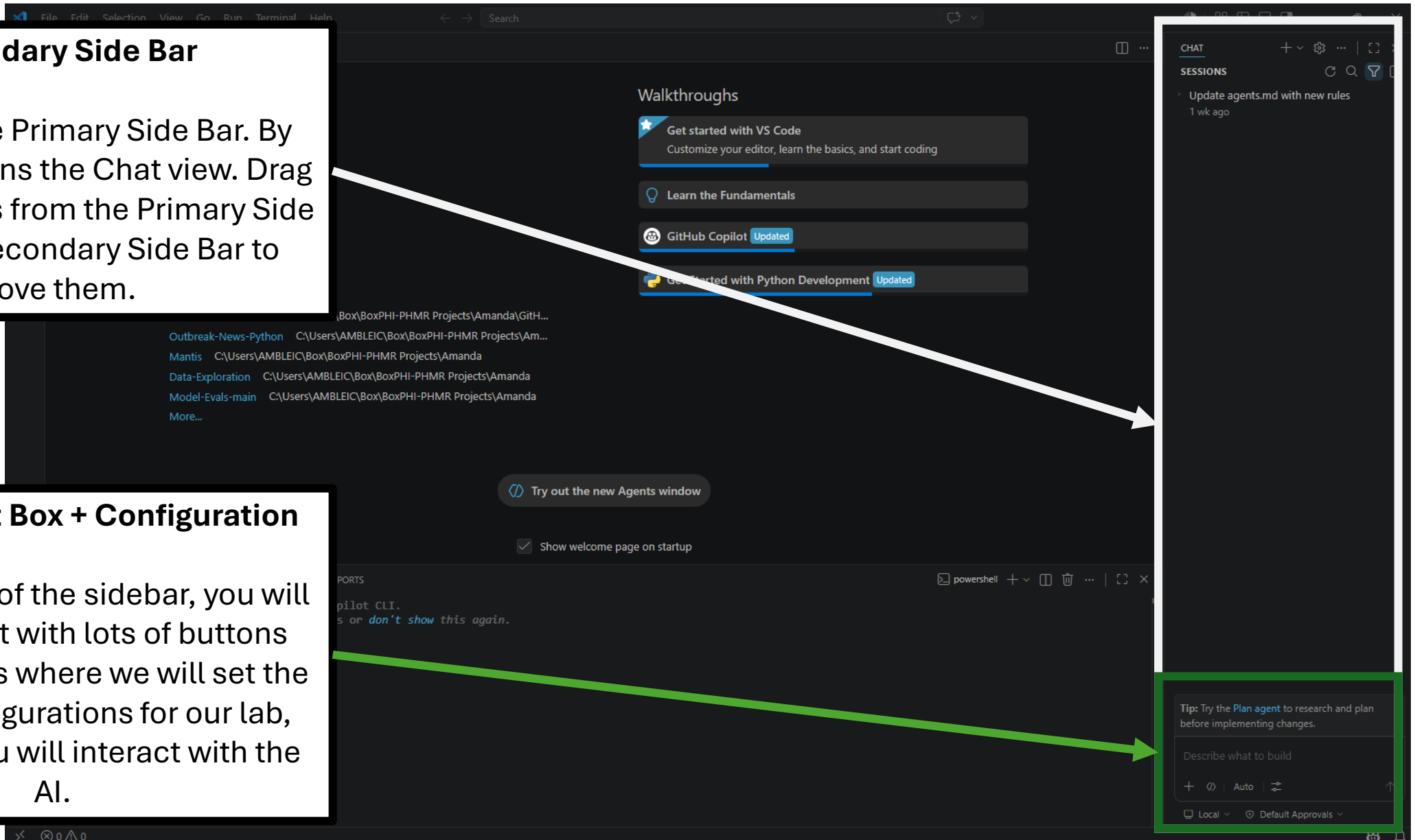
Next to the search bar there is a chat box icon with little stars by it. To open the chat sidebar, click the icon.

Secondary Side Bar

Opposite the Primary Side Bar. By default, contains the Chat view. Drag and drop views from the Primary Side Bar to the Secondary Side Bar to move them.

Primary Chat Box + Configuration

At the bottom of the sidebar, you will see a chatbot with lots of buttons under it. This is where we will set the primary configurations for our lab, and where you will interact with the AI.



Step 2: Choosing an Agent Type

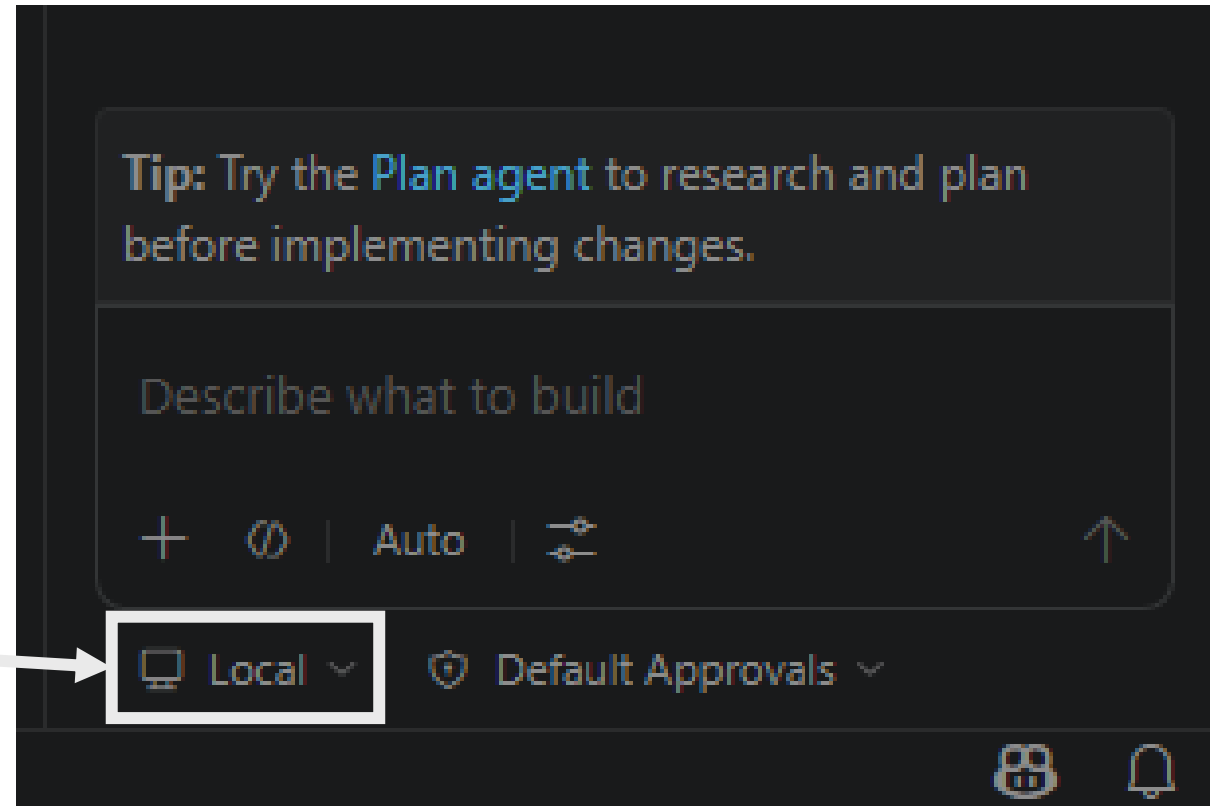
AI agents can operate locally or in the cloud and may function either interactively with user guidance or autonomously in the background. Choosing the appropriate setup depends on the task requirements and desired level of user involvement.

Local: Runs the agent interactively in the editor with full access to your workspace, tools, and models.

Copilot CLI: Runs in the background on your machine, optionally using Git worktrees for isolation.

Cloud: Runs remotely and integrates with GitHub pull requests for team collaboration.

Third-party: Uses the third-party agent harness and SDK from Anthropic and OpenAI to run either locally on your machine or in the cloud.



Want to learn more?

VS Code provides additional information about selecting AI types at:

<https://code.visualstudio.com/docs/copilot/agents/overview>

Step 3: Choosing an Agent

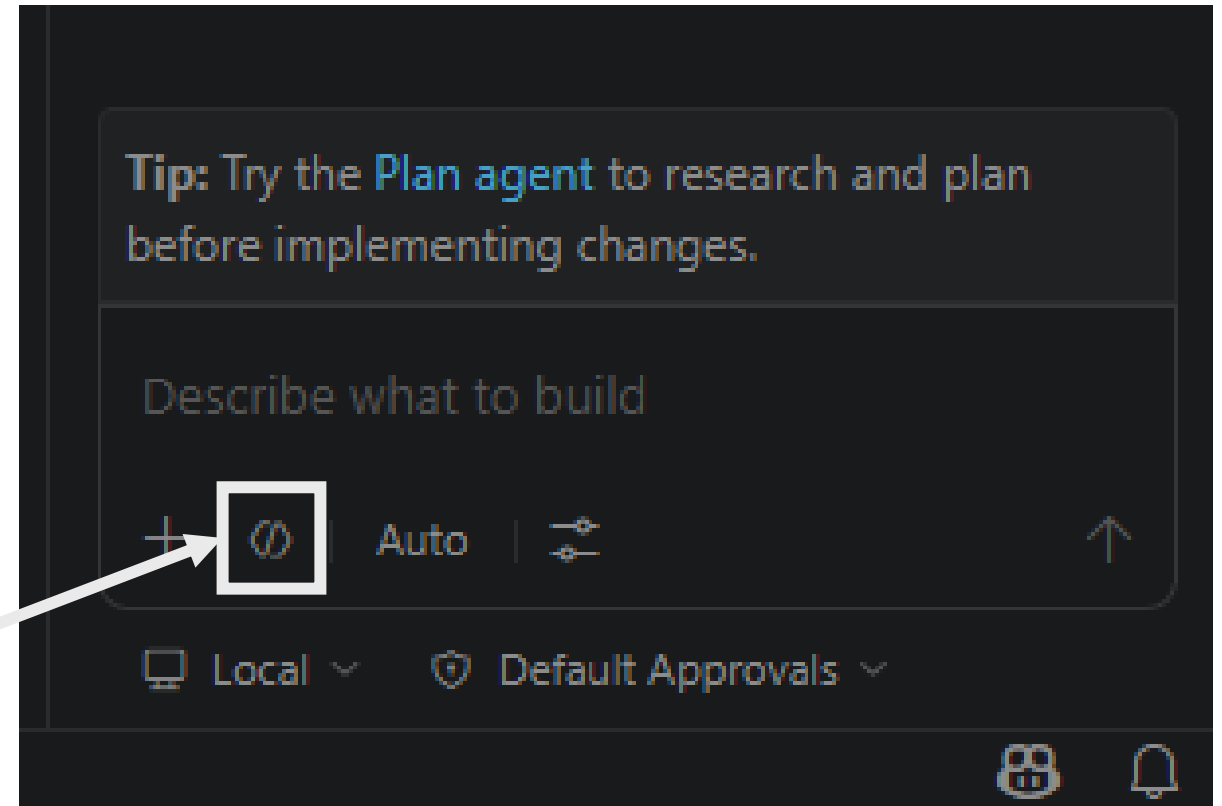
We can select the agent persona we want to use for our tasks. The selected option controls what tools the AI can use and how it completes the task at hand. The three built-in agents are:

Agent: autonomously plans and implements changes across files, runs terminal commands, and invokes tools.

Plan: creates a structured, step-by-step implementation plan before writing any code. Hands the plan off to an implementation agent when it looks right.

Ask: answers questions about coding concepts, your codebase, or VS Code itself without making file changes.

We will be using a mix of all three types throughout the lab. We will start with the 'Agent' option.



Want to learn more?

VS Code provides additional information about choosing agents at:

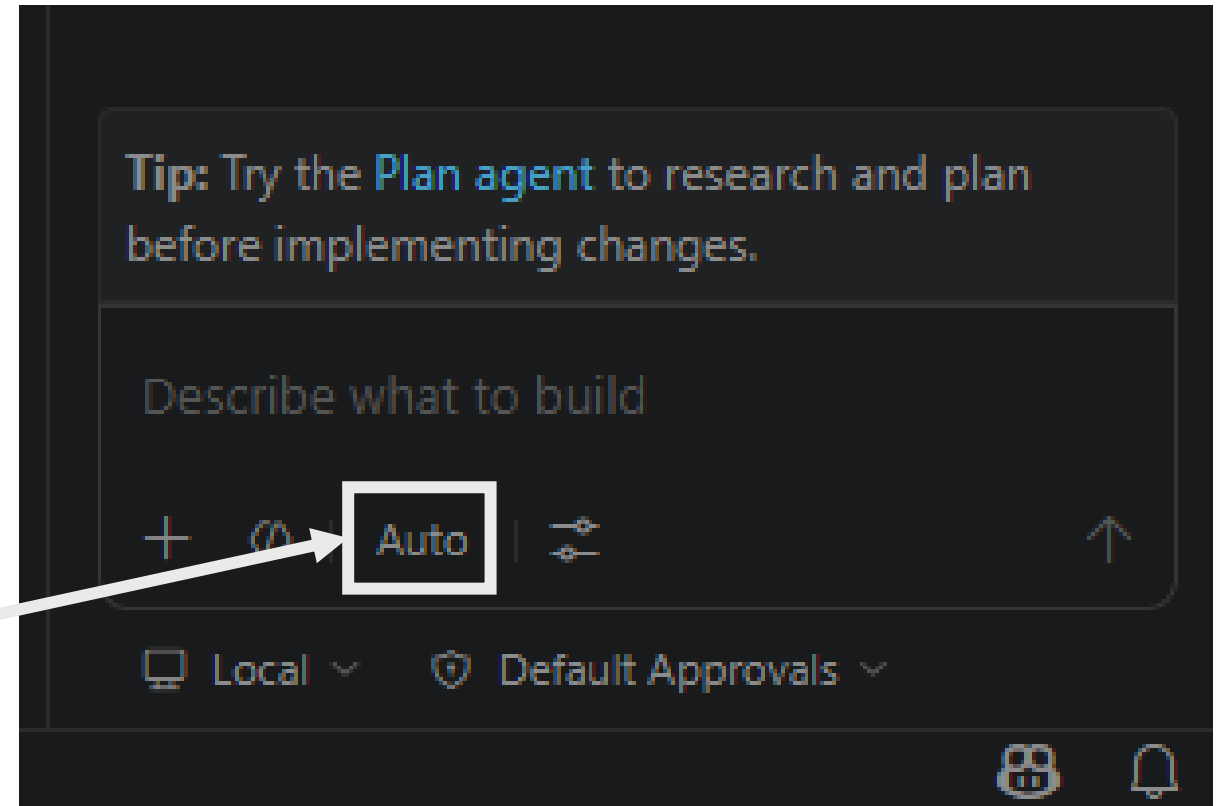
https://code.visualstudio.com/docs/copilot/agents/overview#_choose-an-agent

Step 4: AI Language Models in VS Code

There are a multitude of built-in models available via GitHub Copilot to choose from.

Each model has its own strengths and weaknesses, and choosing the right model is an important piece of working with AI. For example, if your task is simple edits or questions, you can use a “fast” model. For more complex tasks, such as refactoring or multi-step tasks, a reasoning model may be better.

For our lab, we will stick with “Auto” model selection. When “Auto” is selected, VS Code determines a task’s complexity and real-time model availability to select the optimal model.



Want to learn more?

VS Code provides additional information about AI language models in VS Code at:

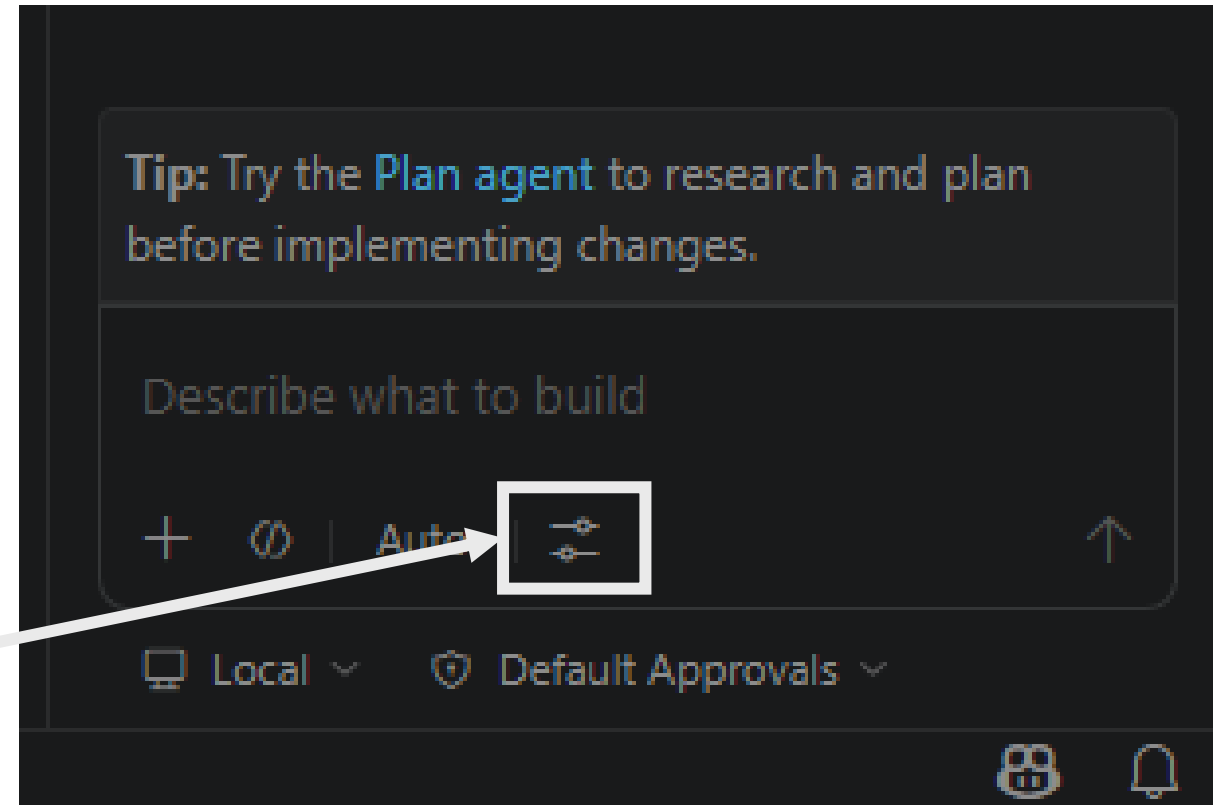
<https://code.visualstudio.com/docs/copilot/customization/language-models>

Step 5: Use Tools with Agents

Tools provide agents specialized functionality for completing specific tasks like searching code, running commands, fetching web content, or invoking APIs.

In VS Code, the agent automatically determines which tools to use from the enabled tools based on your prompt and the context of your request. The agent autonomously chooses and invokes relevant tools as needed to accomplish the task.

For the lab, we will not change any settings related to tools configuration and use the built-in settings.



Want to learn more?

VS Code provides additional information about using tools with agents at:

<https://code.visualstudio.com/docs/copilot/agents/agent-tools>

Step 6: Adding Context to Your Prompts

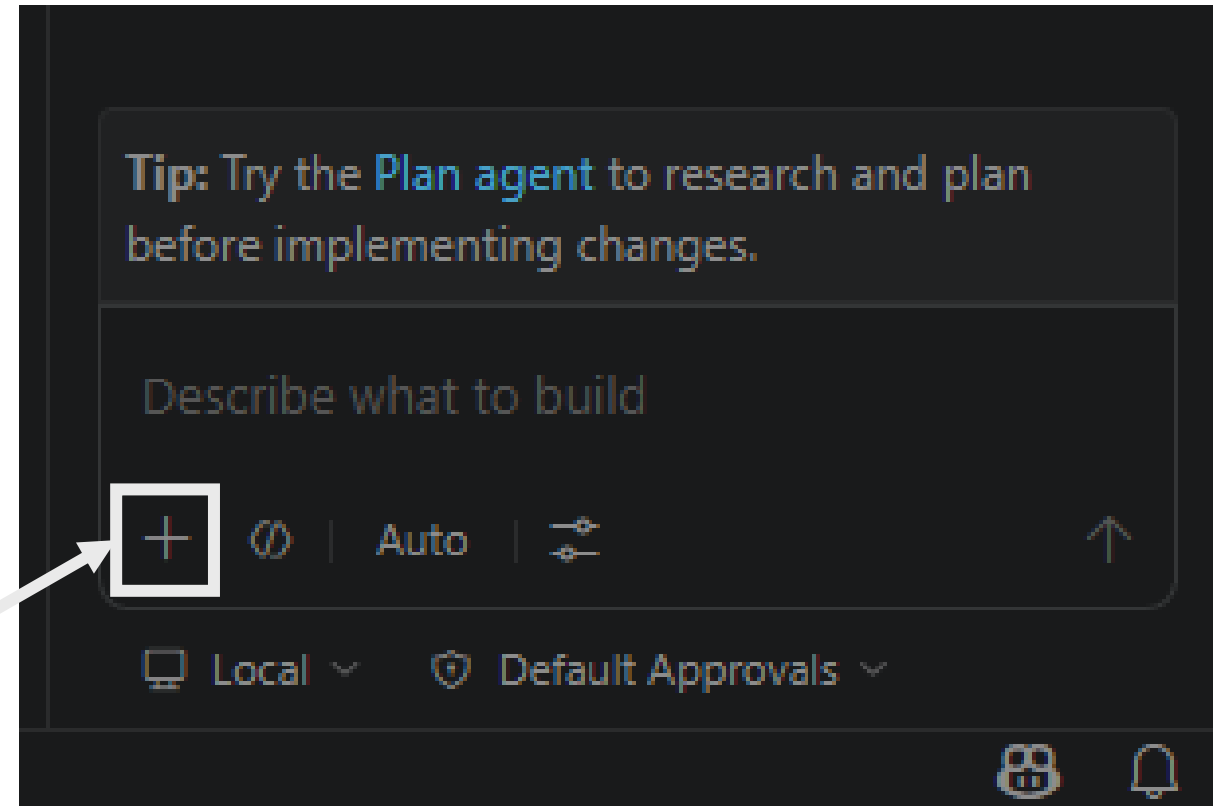
Providing the right context helps the AI generate more relevant and accurate responses.

Implicit context: VS Code automatically includes the active file, your current selection, and the file name as context. When you use agents, the agent decides autonomously if additional context is needed.

#-mentions: type # in the chat input to explicitly reference files (#file), folders, symbols, your codebase (#codebase), terminal output (#terminalSelection), or tools like #fetch.

@-mentions: type @ to invoke specialized chat participants like @vscode or @terminal, each optimized for their respective domain.

Vision: attach images, such as screenshots or UI mockups, as context for your prompt.



Want to learn more?

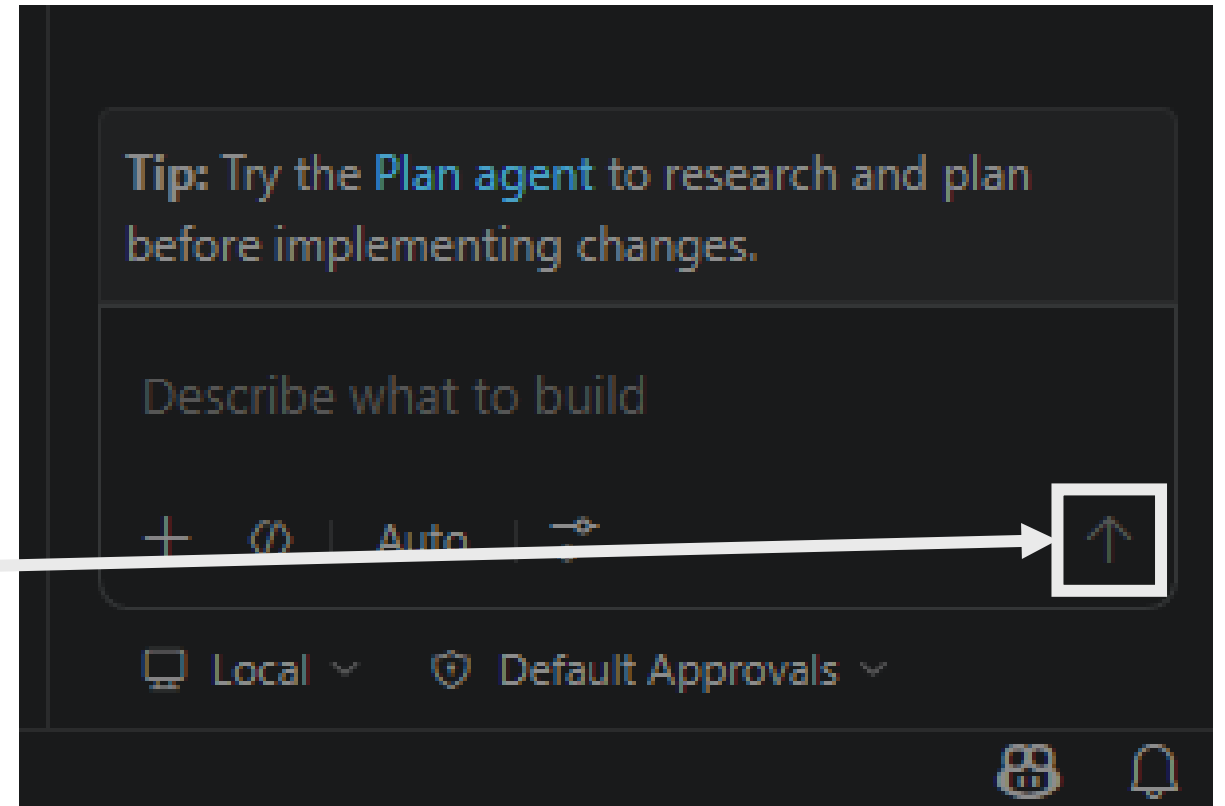
VS Code provides additional information about managing context for AI at:

<https://code.visualstudio.com/docs/copilot/chat/copilot-chat-context>

Step 7: Writing the Prompt & Sending

After setting up the AI configuration, you can now interact with your agent.

Once you have typed your message and attached any necessary context, click the up arrow to send the message.



Want to learn more?

VS Code provides additional information about GitHub Copilot at:

<https://code.visualstudio.com/docs/copilot/overview>



Task 3: Connecting GitHub & VS Code

bleicham-tester / SISMID-2026

Type to search

<> Code Pull requests Actions Projects Wiki Security and quality Insights Settings

SISMID-2026 Public

forked from bleicham/SISMID-2026

main 1 Branch 0 Tags

Go to file T Add file

This branch is up to date with bleicham/SISMID-2026:main

Contribute Sync fork

bleicham Delete Lab-Materials/students/Amanda/scripts directory 5861d18 · 2 weeks ago 6 Commits

Lab-Materials/data Delete Lab-Materials/students/Amanda/scripts directory 2 weeks ago

2 weeks ago

2 weeks ago

for project.

Add a README

About

No description, website, or topics provided.

Activity

0 stars

0 watching

0 forks

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Contributors

No contributors

Step 1: Clicking Code

We are going to clone our fork in VS Code. This will allow us to easily make changes locally and then “push” to GitHub in later steps. First, we need to obtain the HTTPS URL for our GitHub fork. To do so, we first need to **click the green code button**.

Step 2: Copy the HTTPS URL

Next, we will want to copy the HTTPS URL. This will be used within VS Code to connect it to our GitHub fork. To copy, **click** the two overlapping **squares icon**.

SISMID-2026 Public

forked from [bleicham/SISMID-2026](#)

main 1 Branch 0 Tags

This branch is up to date with [bleicham/SISMID-2026:main](#).

bleicham Delete Lab-Materials/students/Amanda/scripts directory

Lab-Materials/data

Delete Lab-Materials

commit



a README

s repository understand your project.

Add a README

Go to file

Add file

<> Code

Local

Codespaces

Clone

HTTPS SSH GitHub CLI

<https://github.com/bleicham-tester/SISMID-2026>

Clone using the web URL

Open with GitHub Desktop

Download ZIP

About

No description, website, or topics provided.

Activity

0 stars

0 watching

0 forks

Releases

No releases published

[Create a new release](#)

Packages

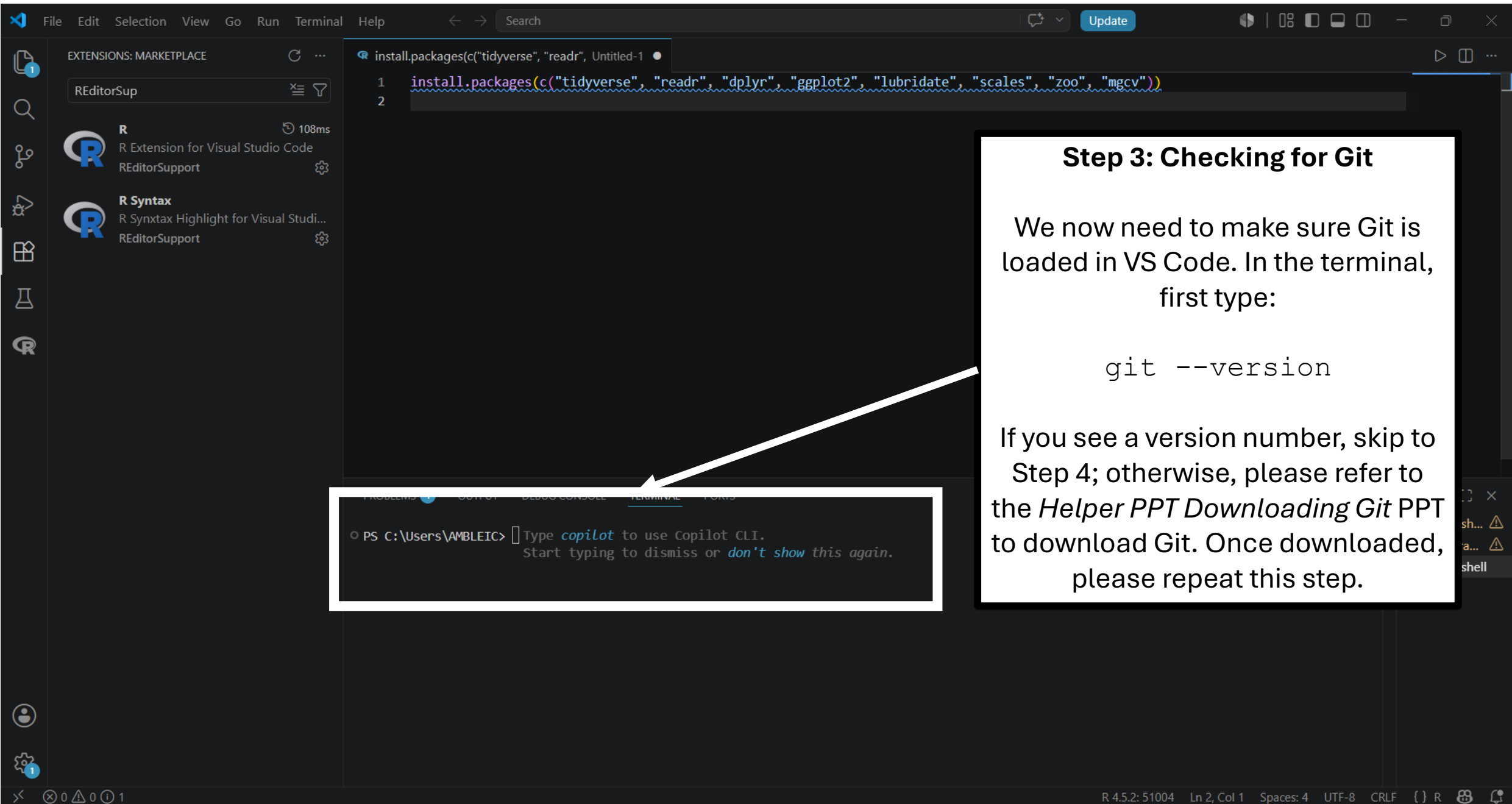
No packages published

[Publish your first package](#)

Contributors

No contributors

Now back to VS Code...



Step 3: Checking for Git

We now need to make sure Git is loaded in VS Code. In the terminal, first type:

```
git --version
```

If you see a version number, skip to Step 4; otherwise, please refer to the *Helper PPT Downloading Git* PPT to download Git. Once downloaded, please repeat this step.

```
PS C:\Users\AMBLEIC> Type copilot to use Copilot CLI.
Start typing to dismiss or don't show this again.
```

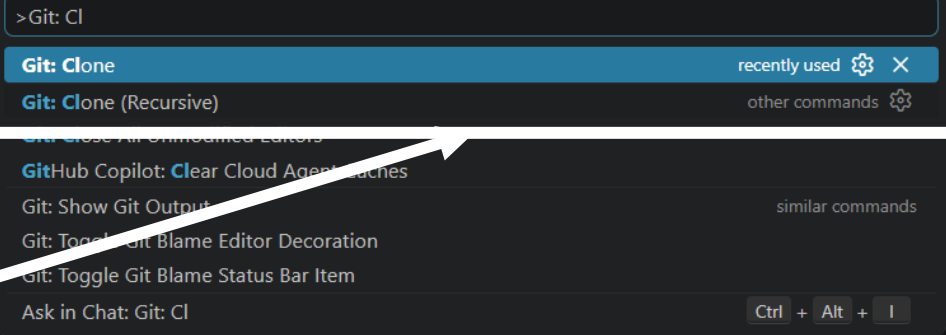
Step 4: Cloning the Repo

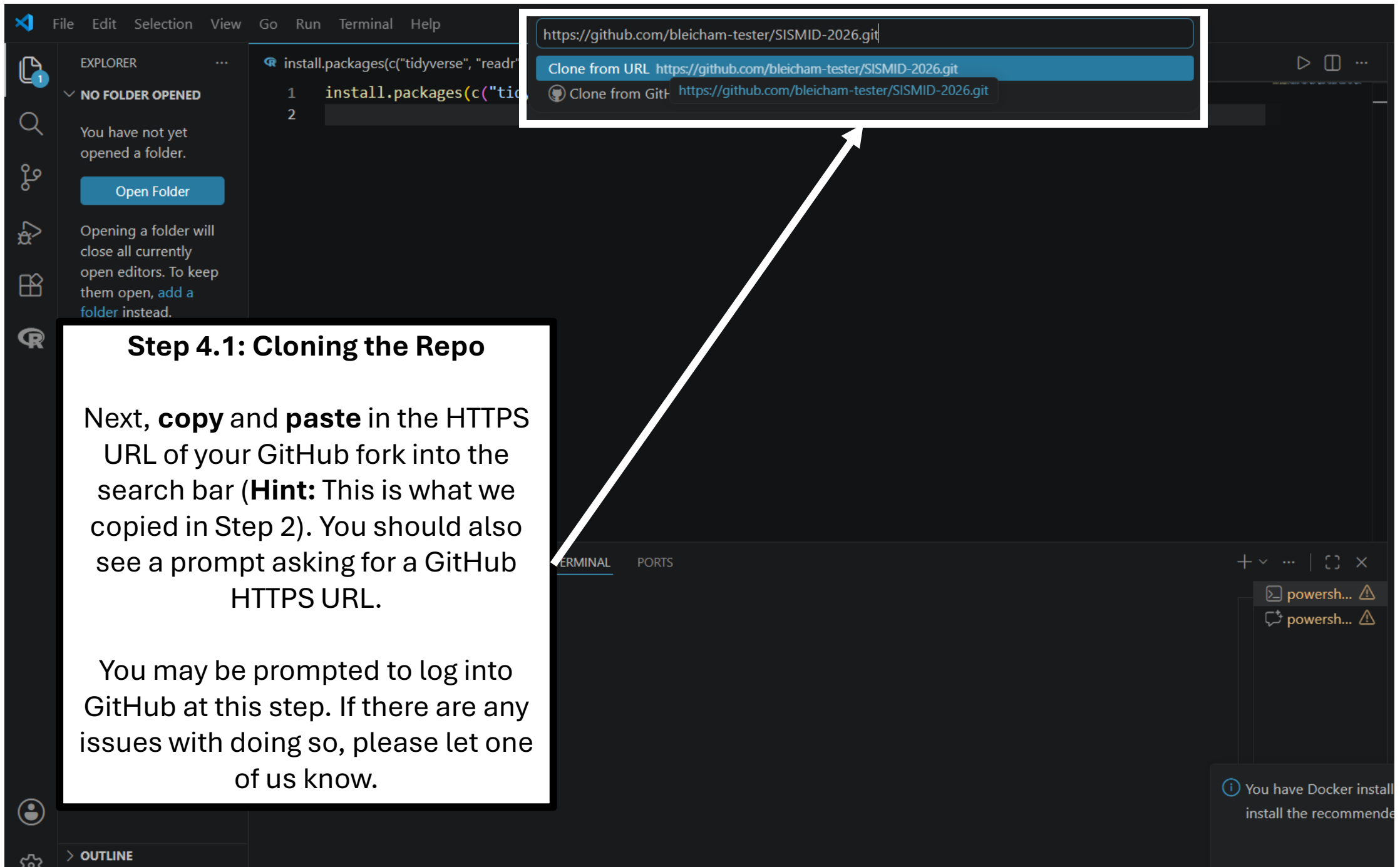
To begin the process of cloning the repo, first click:

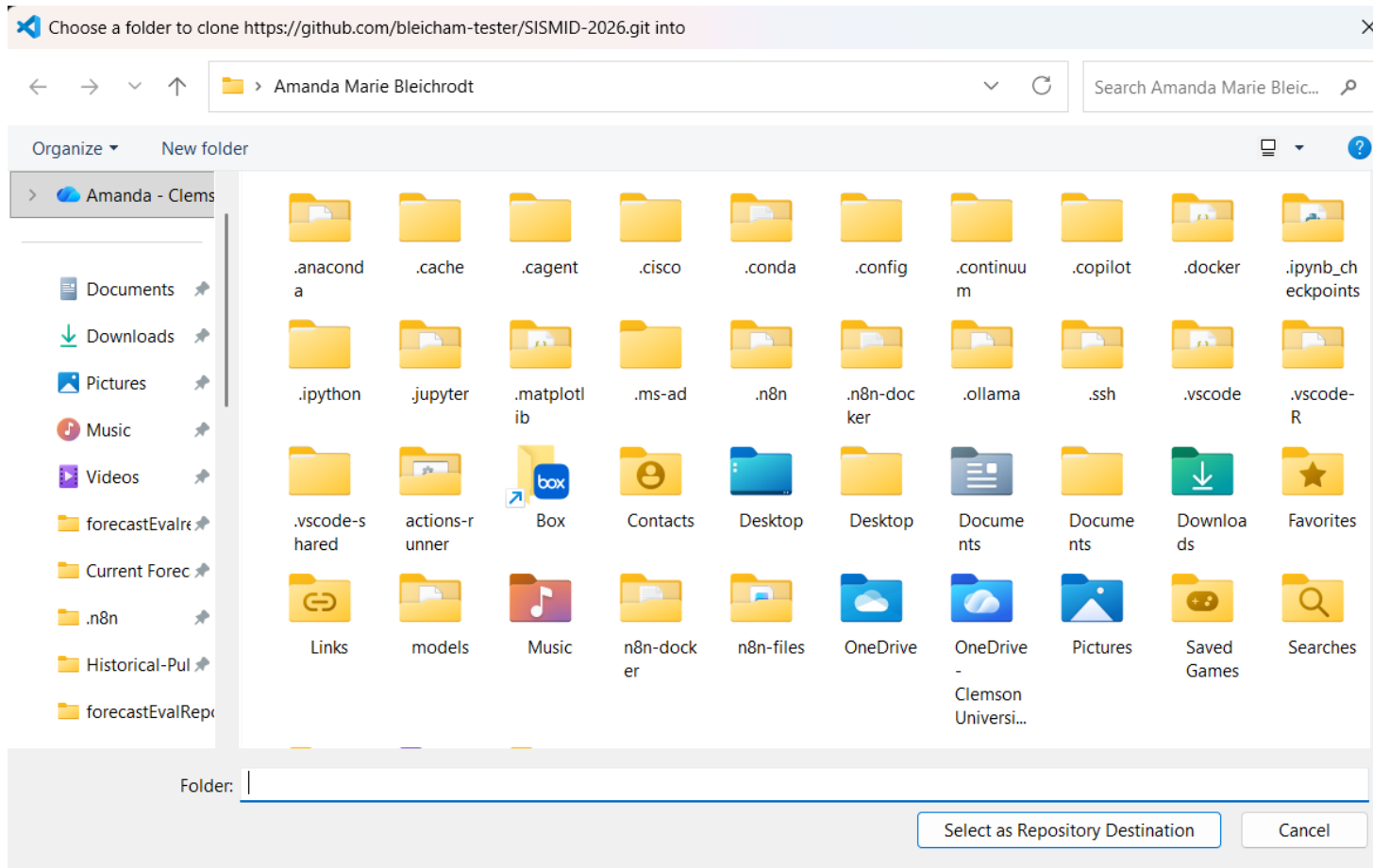
Ctrl + Shift + P

to open the command search at the top of the page. You can also click *View* (top of page) and then select *Command Palette* to open the search. Then type and click

Git: Clone

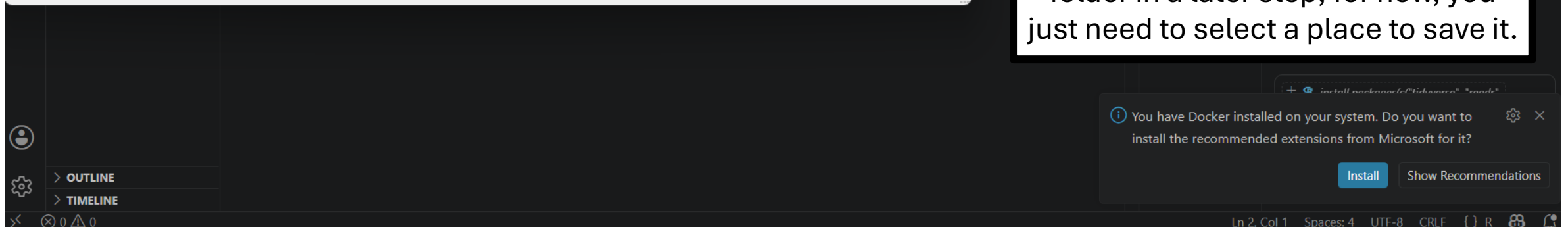






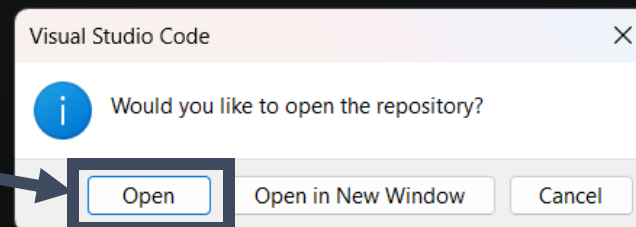
Step 5: Saving the Clone

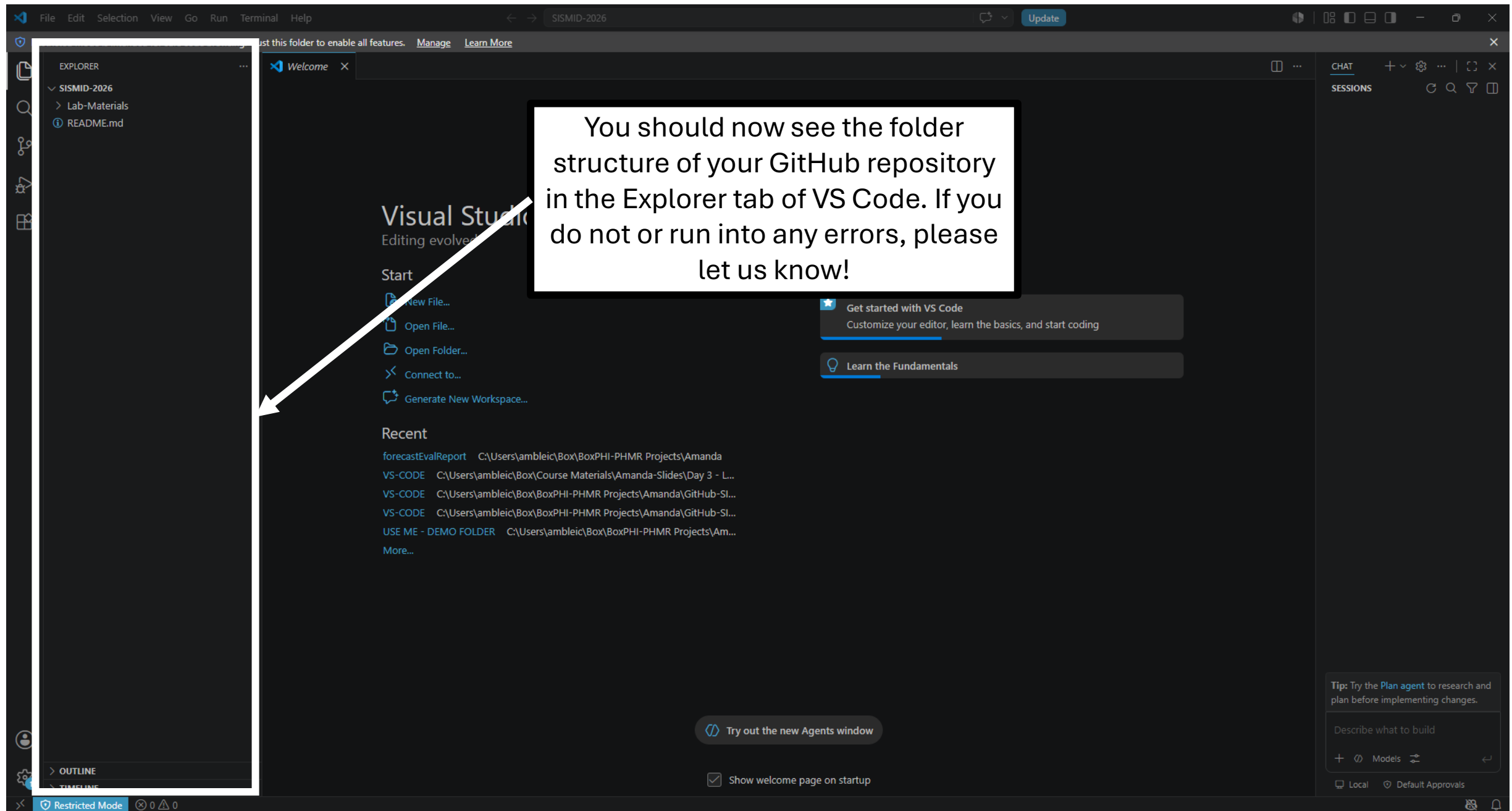
After copy-and-pasting the HTTPS URL, you should see your folder directory. This is where you need to choose a folder to save your GitHub fork into. It will act as the local intermediary between VS Code and what you have on GitHub. We will talk about the structure of this folder in a later step; for now, you just need to select a place to save it.

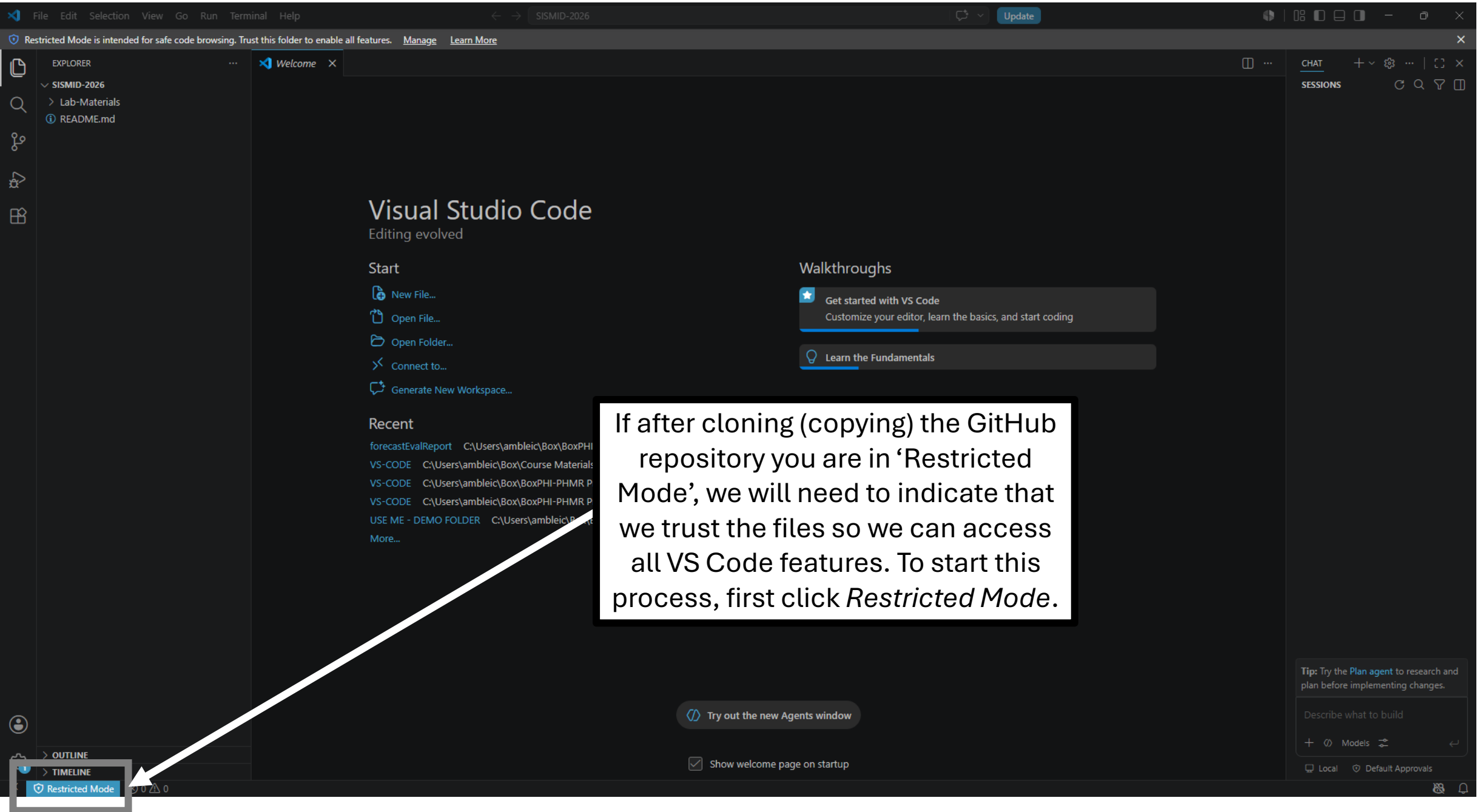


Step 6: Opening the Repository

After choosing a folder to save the local version of the repository, VS Code will prompt you to open it. Here, you want to click **Open**. This will ensure all your working directories are set up correctly and get you ready for the remainder of the lab.









You are in Restricted Mode

Code is in a restricted mode intended for safe code browsing.
[Configure your settings](#) or [learn more](#).

In a Trusted Folder

You trust the authors of the files in the current folder. All features are enabled:

- ✓ Tasks are allowed to run
- ✓ Debugging is enabled
- ✓ All workspace settings are applied
- ✓ All enabled extensions are activated

Trust

In Restricted Mode

You do not trust the authors of the files in the current folder. The following features are disabled:

- ✗ Tasks are not allowed to run
- ✗ Debugging is disabled

Next, click *Trust* and close out the pop-up.

Trusted Folders & Workspaces

You trust the following folders, their subfolders, and workspace files.

Host	Path
Local	C:\Users\ambleic\Box\BoxPHI-PHMR Projects\Amanda\forecastEvalReport
Local	C:\Users\ambleic\Box\BoxPHI-PHMR Projects\Amanda\GitHub-SISMID
Local	C:\Users\ambleic\Box\BoxPHI-PHMR Projects\Amanda\Model-Evals-main
Local	C:\Users\ambleic\Box\Course Materials\Amanda-Slides\Day 3 - Lab Materials\Activity 4\VS-CODE

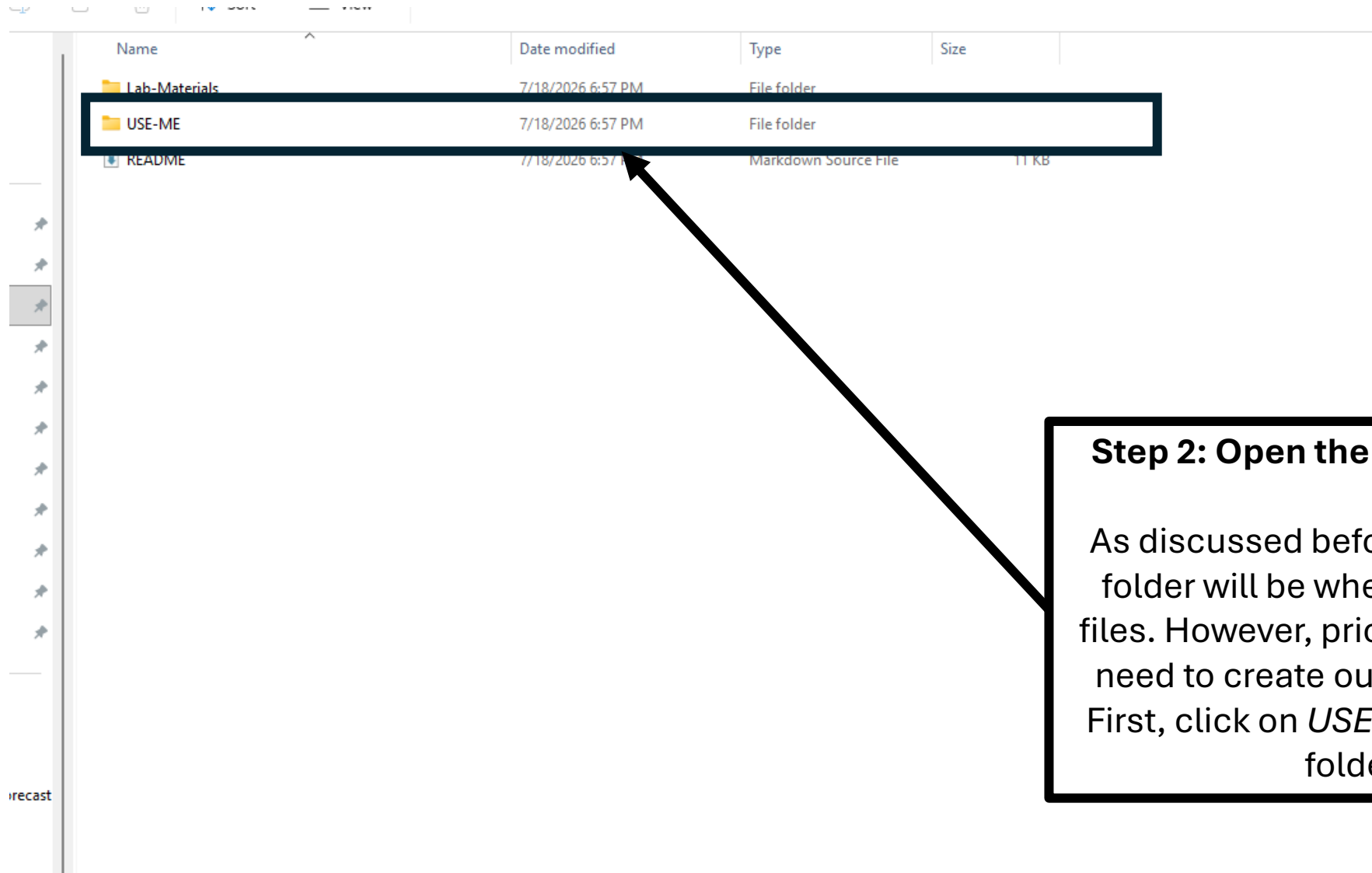


Task 4: Preparing the Working Directory

Name	Date modified	Type	Size
Custom Office Templates	7/12/2026 1:56 AM	File folder	
Zoom	7/17/2026 12:06 PM	File folder	
.Rhistory	7/13/2026 9:22 AM	RHISTORY File	0 KB
SISMID-2026	7/18/2026 6:57 PM	File folder	

Step 1: Navigating to Local GitHub

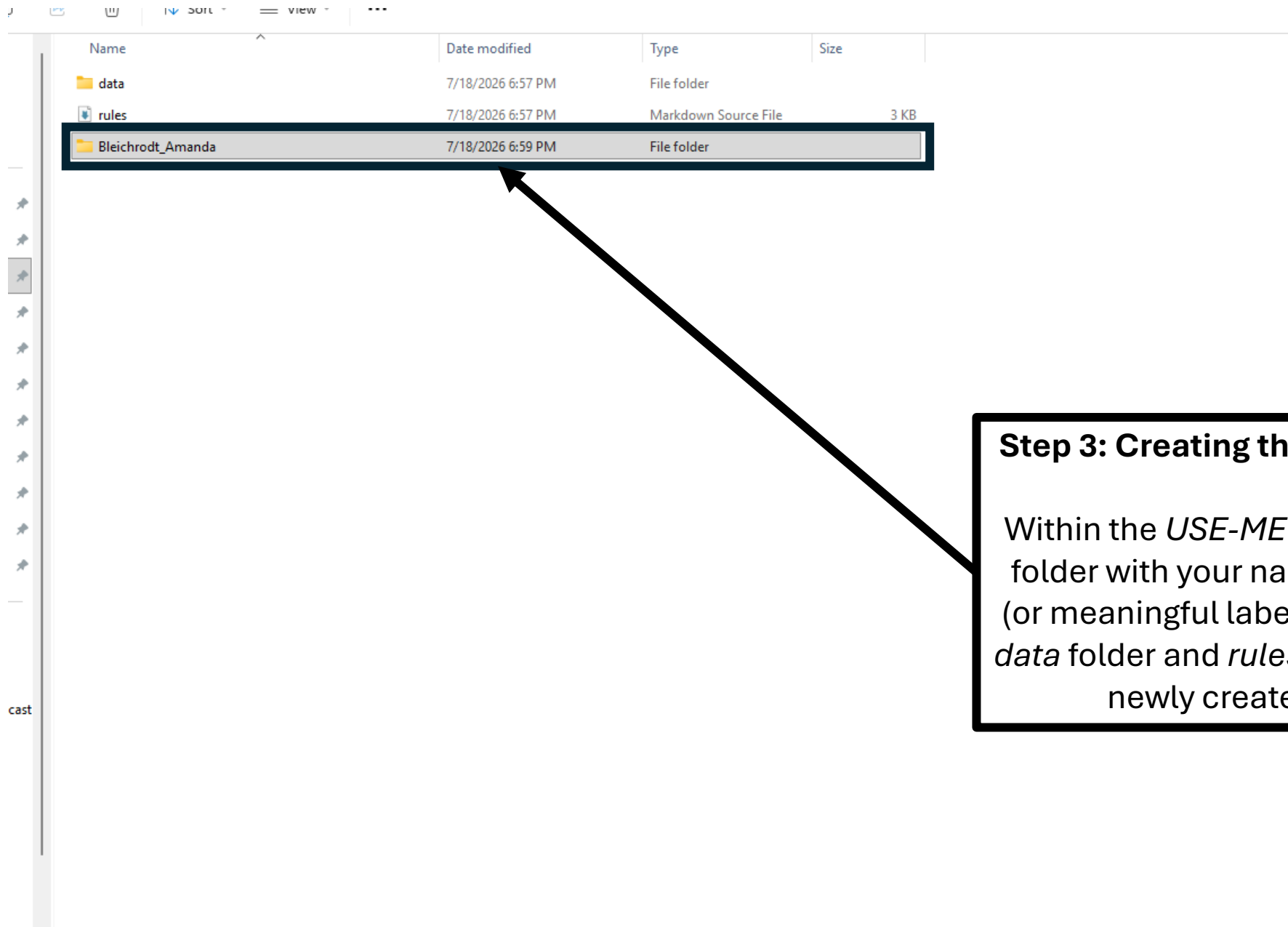
Now that we have cloned our fork (copied the GitHub fork to our computer), we need to do some initial setup. This will come into play when we upload our results to GitHub. **Click** on the downloaded folder.



Name	Date modified	Type	Size
Lab-Materials	7/18/2026 6:57 PM	File folder	
USE-ME	7/18/2026 6:57 PM	File folder	
README	7/18/2026 6:57 PM	Markdown Source File	11 KB

Step 2: Open the *USE-ME* folder

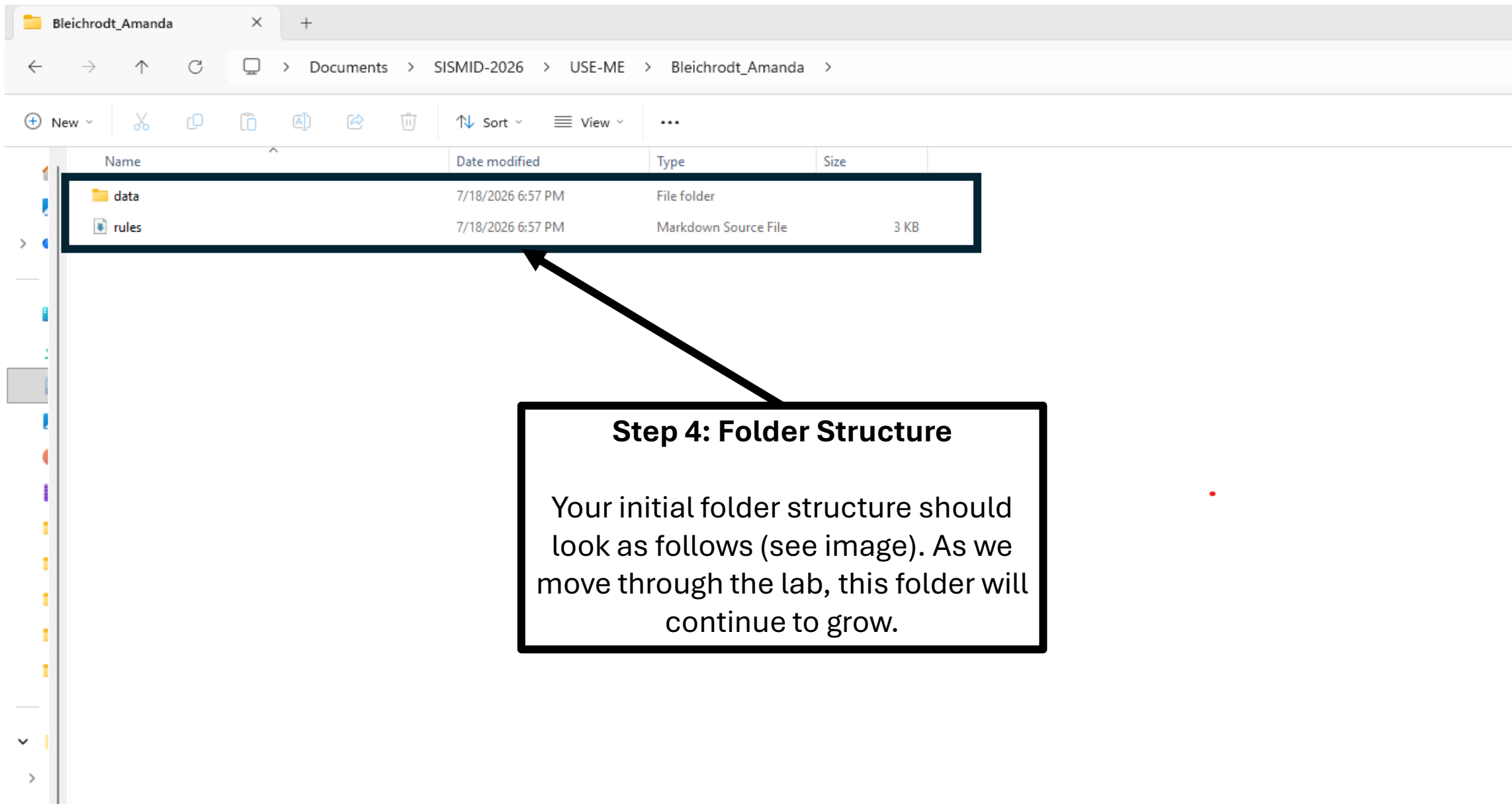
As discussed before, the *USE-ME* folder will be where we save our files. However, prior to doing so we need to create our named folder. First, click on *USE-ME* to open the folder.



Name	Date modified	Type	Size
data	7/18/2026 6:57 PM	File folder	
rules	7/18/2026 6:57 PM	Markdown Source File	3 KB
Bleichrodt_Amanda	7/18/2026 6:59 PM	File folder	

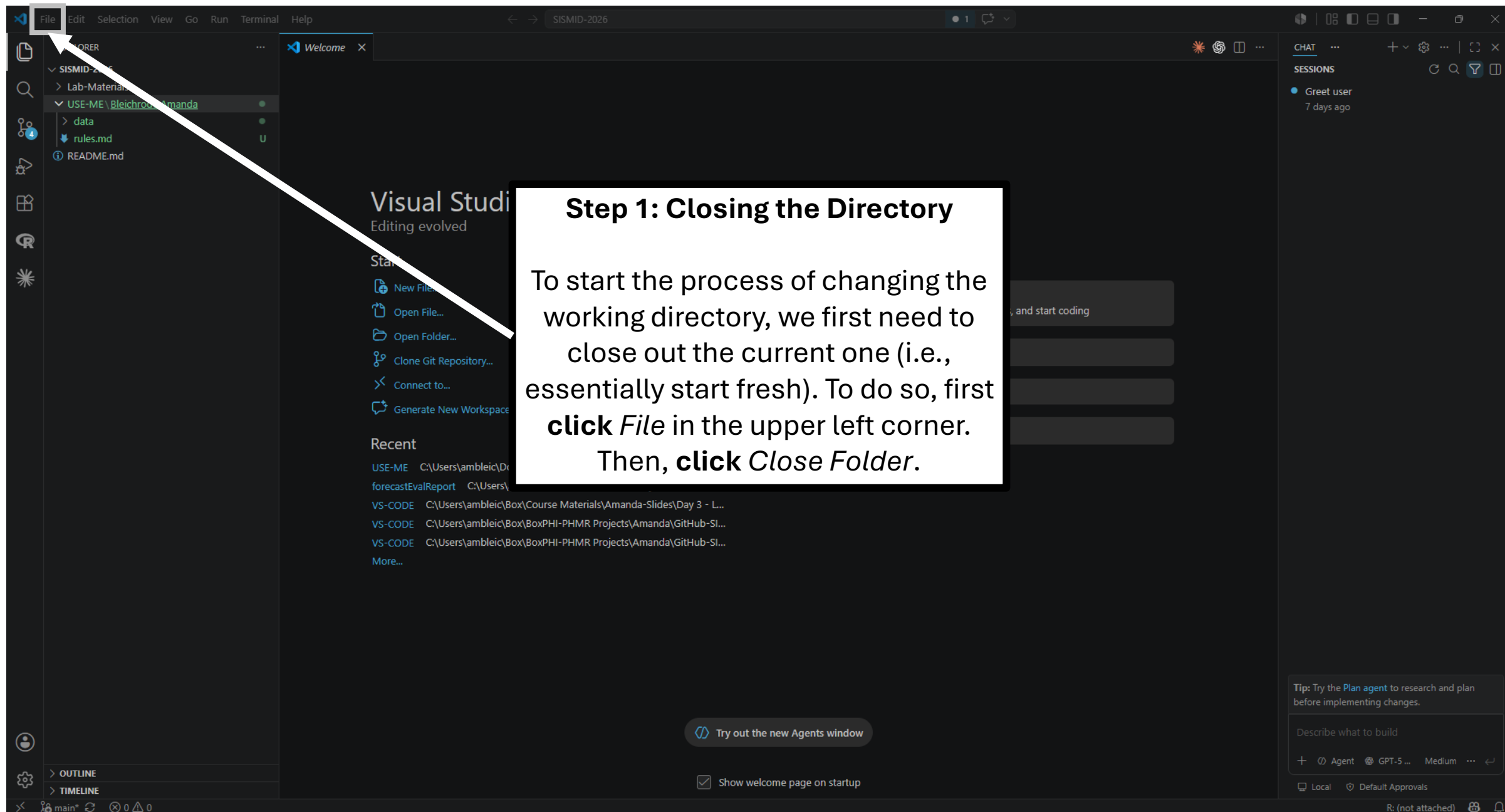
Step 3: Creating the named folder

Within the *USE-ME* folder, **create** a folder with your name as the label (or meaningful label) and **move** the *data* folder and *rules.md* within your newly created folder.



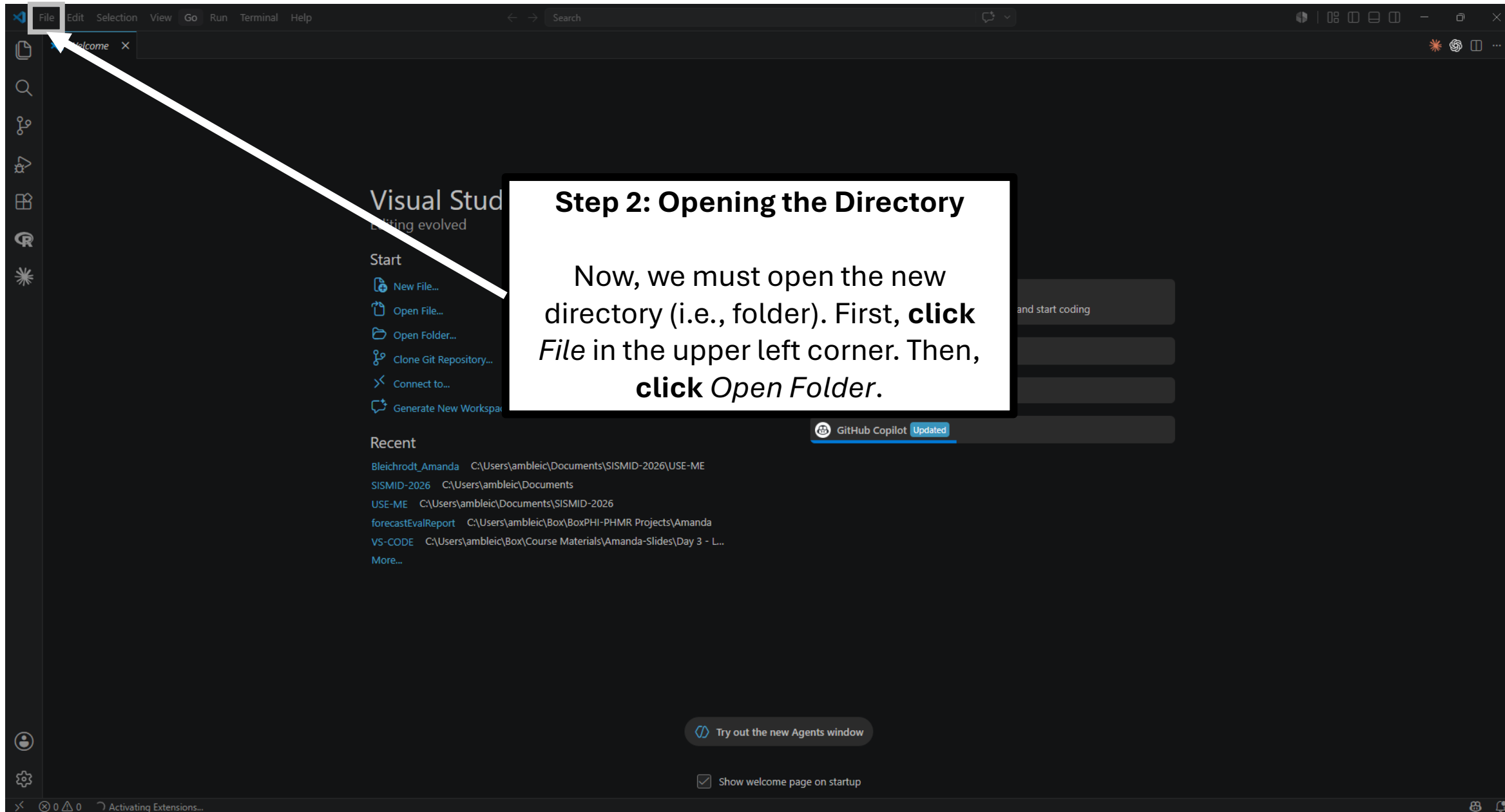


Task 5: Setting the Working Directory in VS Code



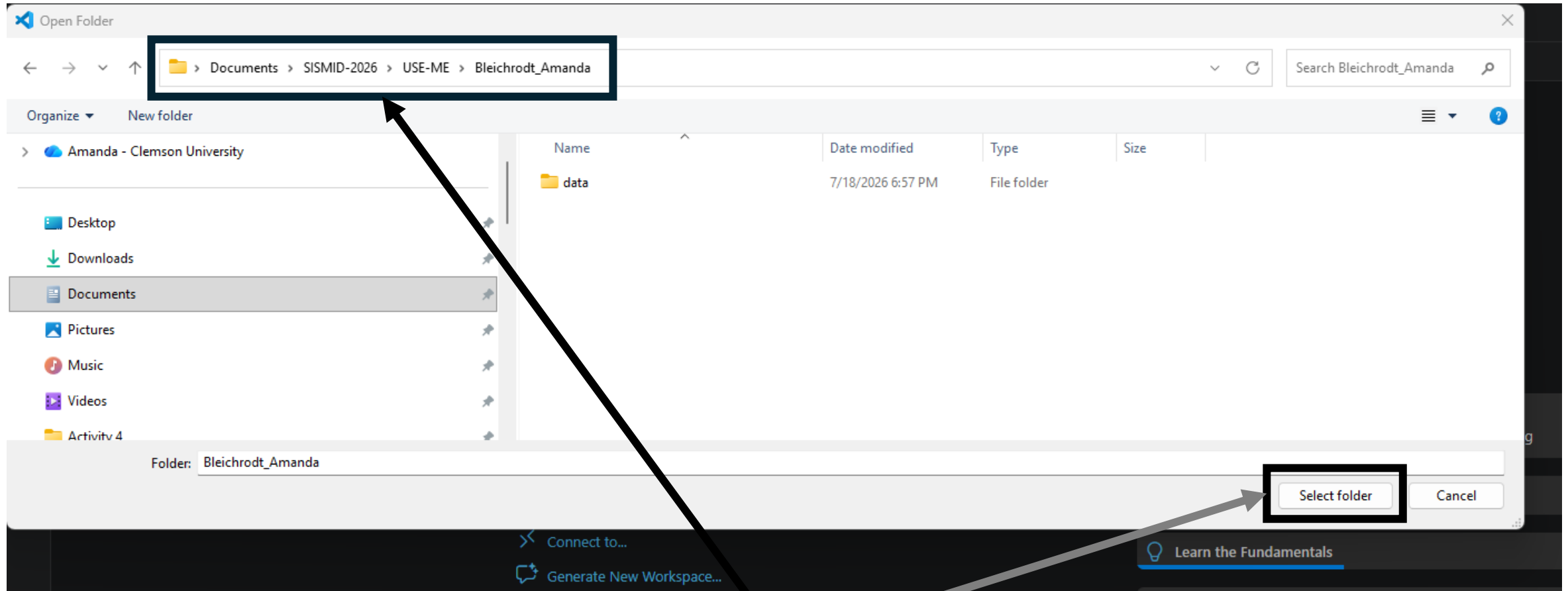
Step 1: Closing the Directory

To start the process of changing the working directory, we first need to close out the current one (i.e., essentially start fresh). To do so, first **click** *File* in the upper left corner. Then, **click** *Close Folder*.



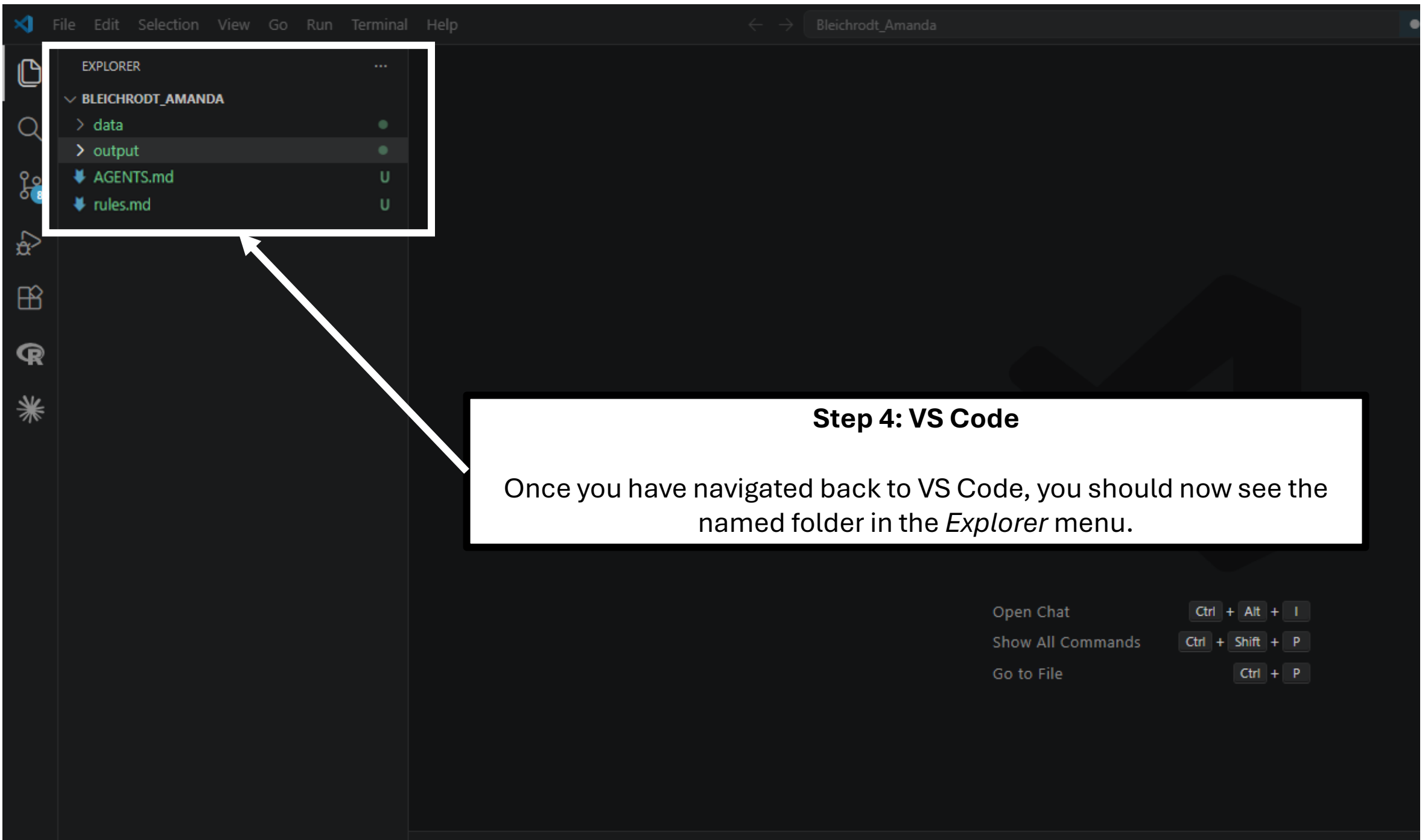
Step 2: Opening the Directory

Now, we must open the new directory (i.e., folder). First, **click File** in the upper left corner. Then, **click Open Folder**.



Step 3: Selecting Your Named Folder

As we are primarily working out of our newly created named folder, we want to select that as our working directory. First, **navigate** to where you saved the GitHub repository and created the named folder. Next, **open** the named folder and click *Select folder*.



Step 4: VS Code

Once you have navigated back to VS Code, you should now see the named folder in the *Explorer* menu.